Capstone: Ovarian Cancer Prediction Simulation

HI 192: Knowledge Representation and Health Decision Support

DELA CRUZ James Angelo **DORIA**, Marc **RODRIGUEZ**, Augustus Clark Raphael

May 2026

Contents

1 Introduction **7**

2 Dataset **8**

2.1 Data Source 8

2.2 Class Distribution 8

2.3 Splitting the Data 8

2.4 Data Integrity 9

3 Preprocessing **10**

3.1 Raw Data Inventory 10

3.2 Normalization 10

3.3 Data Augmentation 10

3.4 Ensuring Experimental Integrity 11

4 Experimental Pipeline **12**

5 Methodology **13**

5.1 Experiment Setup 13

5.2 Architecture and Model Design 13

5.3 Hyperparameters and Optimization 14

5.4 Metrics and Evaluation 14

6 Results **15**

6.1 General Performance 15

6.2 Top Performers 15

6.3 The Impact of Learning Rate 15

6.4 Optimizer Comparison 15

6.5 Visualizing the Results 16

6.6 Failures and Suboptimal Runs 16

7 Discussion **17**

7.1 Why Learning Rate Controlled Everything 17

7.2 Comparing the Optimizers 18

7.3 Architecture Strengths 18

7.4 A Warning about AUC 18

7.5 Recommendations 19

8 Conclusion **20**

9 Appendices **21**
9.1 Per-Model Accuracy and Loss Curves 21
9.2 Per-Model AUC-ROC Curves 30
9.3 Per-Model Confusion Matrices 38
9.4 Complete Metric Suite per Architecture 46
9.4.1 VGG19 46
9.4.2 ResNet50 47
9.4.3 DenseNet121 48
9.4.4 EfficientNetB3 49
9.5 Complete 72-Run Results Table 50
9.6 Source Code 52

List of Figures

1 Experiment Pipeline 12
2 Full Accuracy Heatmap 16
3 Full AUC-ROC Heatmap 17
4 VGG19,Adam: Accuracy and Loss Curves across Learning Rates . . . 22
5 VGG19,Adagrad: Accuracy and Loss Curves across Learning Rates . 22
6 VGG19,Adamax: Accuracy and Loss Curves across Learning Rates . 22
7 VGG19,AdaDelta: Accuracy and Loss Curves across Learning Rates . 23
8 VGG19,SGD: Accuracy and Loss Curves across Learning Rates . . . 23
9 VGG19,RMSProp: Accuracy and Loss Curves across Learning Rates . 23
10 EfficientNetB3,Adam: Accuracy and Loss Curves across Learning Rates 24
11 EfficientNetB3,Adagrad: Accuracy and Loss Curves across Learning Rates 24
12 EfficientNetB3,Adamax: Accuracy and Loss Curves across Learning Rates 24
13 EfficientNetB3,AdaDelta: Accuracy and Loss Curves across Learning Rates 25
14 EfficientNetB3,SGD: Accuracy and Loss Curves across Learning Rates 25
15 EfficientNetB3,RMSProp: Accuracy and Loss Curves across Learning Rates 25
16 ResNet50,Adam: Accuracy and Loss Curves across Learning Rates . . 26
17 ResNet50,Adagrad: Accuracy and Loss Curves across Learning Rates 26
18 ResNet50,Adamax: Accuracy and Loss Curves across Learning Rates 26
19 ResNet50,AdaDelta: Accuracy and Loss Curves across Learning Rates 27
20 ResNet50,SGD: Accuracy and Loss Curves across Learning Rates . . 27
21 ResNet50,RMSProp: Accuracy and Loss Curves across Learning Rates 27
22 DenseNet121,Adam: Accuracy and Loss Curves across Learning Rates 28
23 DenseNet121,Adagrad: Accuracy and Loss Curves across Learning Rates 28
24 DenseNet121,Adamax: Accuracy and Loss Curves across Learning Rates 28
25 DenseNet121,AdaDelta: Accuracy and Loss Curves across Learning Rates 29
26 DenseNet121,SGD: Accuracy and Loss Curves across Learning Rates 29
27 DenseNet121,RMSProp: Accuracy and Loss Curves across Learning Rates 29
28 VGG19,Adam: AUC-ROC Curves across Learning Rates 30
29 VGG19,Adagrad: AUC-ROC Curves across Learning Rates 30
30 VGG19,Adamax: AUC-ROC Curves across Learning Rates 30
31 VGG19,AdaDelta: AUC-ROC Curves across Learning Rates 31
32 VGG19,SGD: AUC-ROC Curves across Learning Rates 31
33 VGG19,RMSProp: AUC-ROC Curves across Learning Rates 31
34 EfficientNetB3,Adam: AUC-ROC Curves across Learning Rates . . . 32

35 EfficientNetB3,Adagrad: AUC-ROC Curves across Learning Rates . . 32
36 EfficientNetB3,Adamax: AUC-ROC Curves across Learning Rates . . 32
37 EfficientNetB3,AdaDelta: AUC-ROC Curves across Learning Rates . 33
38 EfficientNetB3,SGD: AUC-ROC Curves across Learning Rates 33
39 EfficientNetB3,RMSProp: AUC-ROC Curves across Learning Rates . 33
40 ResNet50,Adam: AUC-ROC Curves across Learning Rates 34
41 ResNet50,Adagrad: AUC-ROC Curves across Learning Rates 34
42 ResNet50,Adamax: AUC-ROC Curves across Learning Rates 34
43 ResNet50,AdaDelta: AUC-ROC Curves across Learning Rates 35
44 ResNet50,SGD: AUC-ROC Curves across Learning Rates 35
45 ResNet50,RMSProp: AUC-ROC Curves across Learning Rates 35
46 DenseNet121,Adam: AUC-ROC Curves across Learning Rates 36
47 DenseNet121,Adagrad: AUC-ROC Curves across Learning Rates . . . 36
48 DenseNet121,Adamax: AUC-ROC Curves across Learning Rates . . . 36
49 DenseNet121,AdaDelta: AUC-ROC Curves across Learning Rates . . 37
50 DenseNet121,SGD: AUC-ROC Curves across Learning Rates 37
51 DenseNet121,RMSProp: AUC-ROC Curves across Learning Rates . . 37
52 VGG19,Adam: Confusion Matrices across Learning Rates 38
53 VGG19,Adagrad: Confusion Matrices across Learning Rates 38
54 VGG19,Adamax: Confusion Matrices across Learning Rates 38
55 VGG19,AdaDelta: Confusion Matrices across Learning Rates 39
56 VGG19,SGD: Confusion Matrices across Learning Rates 39
57 VGG19,RMSProp: Confusion Matrices across Learning Rates 39
58 EfficientNetB3,Adam: Confusion Matrices across Learning Rates . . . 40
59 EfficientNetB3,Adagrad: Confusion Matrices across Learning Rates . 40
60 EfficientNetB3,Adamax: Confusion Matrices across Learning Rates . 40
61 EfficientNetB3,AdaDelta: Confusion Matrices across Learning Rates . 41
62 EfficientNetB3,SGD: Confusion Matrices across Learning Rates . . . 41
63 EfficientNetB3,RMSProp: Confusion Matrices across Learning Rates 41
64 ResNet50,Adam: Confusion Matrices across Learning Rates 42
65 ResNet50,Adagrad: Confusion Matrices across Learning Rates 42
66 ResNet50,Adamax: Confusion Matrices across Learning Rates 42
67 ResNet50,AdaDelta: Confusion Matrices across Learning Rates 43
68 ResNet50,SGD: Confusion Matrices across Learning Rates 43
69 ResNet50,RMSProp: Confusion Matrices across Learning Rates . . . 43
70 DenseNet121,Adam: Confusion Matrices across Learning Rates 44
71 DenseNet121,Adagrad: Confusion Matrices across Learning Rates . . 44
72 DenseNet121,Adamax: Confusion Matrices across Learning Rates . . 44
73 DenseNet121,AdaDelta: Confusion Matrices across Learning Rates . . 45

74 DenseNet121,SGD: Confusion Matrices across Learning Rates 45
75 DenseNet121,RMSProp: Confusion Matrices across Learning Rates . 45

List of Tables

1 Breakdown of images by class in the STRAMPN dataset. 8
2 Stratified train, validation, and test split. 9
3 Augmentation settings applied to the training set. 11
4 Classification Head 13
5 Experiment Matrix 14
6 Per-Architecture AUC Ranking 15
7 VGG19 results at learning rate 1e-4 46
8 VGG19 results at learning rate 1e-5 46
9 VGG19 results at learning rate 1e-6 46
10 ResNet50 results at learning rate 1e-4 47
11 ResNet50 results at learning rate 1e-5 47
12 ResNet50 results at learning rate 1e-6 47
13 DenseNet121 results at learning rate 1e-4 48
14 DenseNet121 results at learning rate 1e-5 48
15 DenseNet121 results at learning rate 1e-6 48
16 EfficientNetB3 results at learning rate 1e-4 49
17 EfficientNetB3 results at learning rate 1e-5 49
18 EfficientNetB3 results at learning rate 1e-6 49
19 Complete 72-run results table sorted by AUC-ROC descending. . . . 51

1 Introduction

Ovarian cancer is one of the most serious challenges in gynecological oncology, often diagnosed at late stages where the prognosis is poor. Early and accurate detection is critical, and histopathology remains the gold standard for diagnosis. However, manually analyzing these slides is time-consuming and depends heavily on the expert's experience. With the rise of deep learning, there is a lot of interest in using computer vision to help automate this process, especially with transfer learning which allows us to use powerful models pre-trained on massive datasets like ImageNet even when we have limited medical data.

In this project, we wanted to see how different optimization settings affect the performance of these models when applied to histopathological images. Instead of just picking one model and one optimizer, we conducted a systematic study of 72 different configurations. We combined four well-established architectures, namely VGG19, EfficientNetB3, ResNet50, and DenseNet121, with six different optimizers and three learning rates. The goal was to find which settings are the most reliable for this specific task and to understand why some configurations fail while others succeed.

This paper is organized as follows: we first describe the STRAMPN dataset and our preprocessing steps. Then, we outline our experimental setup and the specific architectures and optimizers we tested. Finally, we present our results, discuss the trade-offs we observed between different settings, and conclude with recommendations for future research in automated cancer classification.

2 Dataset

2.1 Data Source

For this study, we used the **STRAMPN Histopathological Images for Ovarian Cancer Prediction** dataset from IEEE DataPort. It can be found at:

https://ieee-dataport.org/documents/strampn-histopathological-images-ovarian-cancer-prediction

The dataset contains histopathological JPEG images of ovarian tissue, labeled as either *Ovarian Cancer* or *Ovarian Non-Cancer*.

2.2 Class Distribution

There are 987 images in total. As shown in Table 1, the dataset is reasonably balanced.

Table 1: Breakdown of images by class in the STRAMPN dataset.

Class	Count	Percentage
Ovarian Cancer	481	48.73%
Ovarian Non-Cancer	506	51.27%
Total	**987**	**100%**

Since the split between cancer and non-cancer images is nearly 50/50 there is little concern in regards to class imbalance. Allowing accuracy and AUC to remain the primary metrics of focus in the simulation, with the a suite of metrics collected alongside the primary two (accuracy, recall, specificity, F1-score)

2.3 Splitting the Data

As specified, the dataset was split into training, validation, and test sets using a two-stage process. First, we took 75% of the images from each class for our training and validation pool, leaving 25% for the final test set. Then, we split that 75% pool again, using 70% for actual training and 30% for validation.

We used `sklearn`'s `train_test_split` with `stratify=labels` to make sure the class proportions stayed the same in every subset. We also fixed the `random_state` to 42 so that our splits are completely reproducible. The final counts for each split are in Table 2.

Table 2: Stratified train, validation, and test split.

Split	Cancer	Non-Cancer	Total	% of Dataset
Train	253	265	518	52.48%
Validation	108	114	222	22.49%
Test	120	127	247	25.03%
Total	**481**	**506**	**987**	**100%**

By keeping the seed at 42, we ensured that every one of our 72 training runs used the exact same images for training and testing.

2.4 Data Integrity

After copying the files into their respective folders, we ran a quick check in our preprocessing script to make sure no images were accidentally placed in more than one split (data leakage). Our script confirmed that there were no overlaps:

```
PREPROCESSING COMPLETE: No integrity violations found
```

This gives us confidence that our evaluation results are based on truly unseen data.

3 Preprocessing

We handled all our preprocessing using a custom script, `src/preprocessing.py`. We used standard libraries like `pathlib` and `sklearn`, keeping this step separate from the training loop to ensure the data preparation was clear and repeatable.

3.1 Raw Data Inventory

First, we took an inventory of the raw dataset to make sure all files were valid JPEGs and that we had the expected counts for each class. The script confirmed that we had 481 cancer images and 506 non-cancer images, all in the correct format.

```
------------------------------------------------------------
STEP 1: RAW DATASET INVENTORY
------------------------------------------------------------
  [cancer]     Total files: 481  [ok]
  [no_cancer]  Total files: 506  [ok]
  Total images: 987  [ok]
```

3.2 Normalization

We applied **samplewise standard normalization** to every image using Keras's `ImageDataGenerator`. This involved:

- Centering each image by subtracting its mean pixel value.
- Standardizing each image by dividing by its standard deviation.

Samplewise normalization was chosen for its reliability for relatively small datasets such at this one. As opposed to relying on global statistics (which can be noisy or lead to data leakage), each image is processed independently. This makes the evaluation more reliable and ensures the model handles new images correctly during testing.

3.3 Data Augmentation

To help the models learn more robust features and avoid overfitting, we used data augmentation, but **only for the training set**. We kept the validation and test sets in their original state (except for normalization) to ensure the results reflect real-world performance.

Table 3 summarizes the augmentations we used. We picked these to simulate the typical variations found in histology slides, such as different rotations and slight scaling differences.

Table 3: Augmentation settings applied to the training set.

Setting	Value	Goal
`rotation_range`	30°	Handle slides at any angle
`width/height shift`	0.15	Account for slight centering variations
`shear_range`	0.3	Simulate tissue distortion
`zoom_range`	0.30	Handle magnification differences
`horizontal_flip`	True	Tissue features are orientation-independent

3.4 Ensuring Experimental Integrity

A key part of our methodology was ensuring that the data generators were only initialized **once** at the start of the entire 72-run simulation. If we had recreated the generators inside the training loop, each model would have seen the training images in a different order, adding "luck of the draw" as a hidden factor in our results. By sharing the same generators, we ensured that every architecture and optimizer was tested on the exact same sequence of data.

4 Experimental Pipeline

The pipeline was designed to be as consistent as possible across all 72 configurations. We wanted to make sure that the only things changing were the architecture, optimizer, and learning rate all according to specification. The process follows a clear path from raw data to final evaluation.

The diagram in Figure 1 shows the high-level flow. Starting by preparing the data (splitting and augmenting), then move into the core training loop where we iterate through our experimental matrix, and finally aggregate all the metrics for analysis.

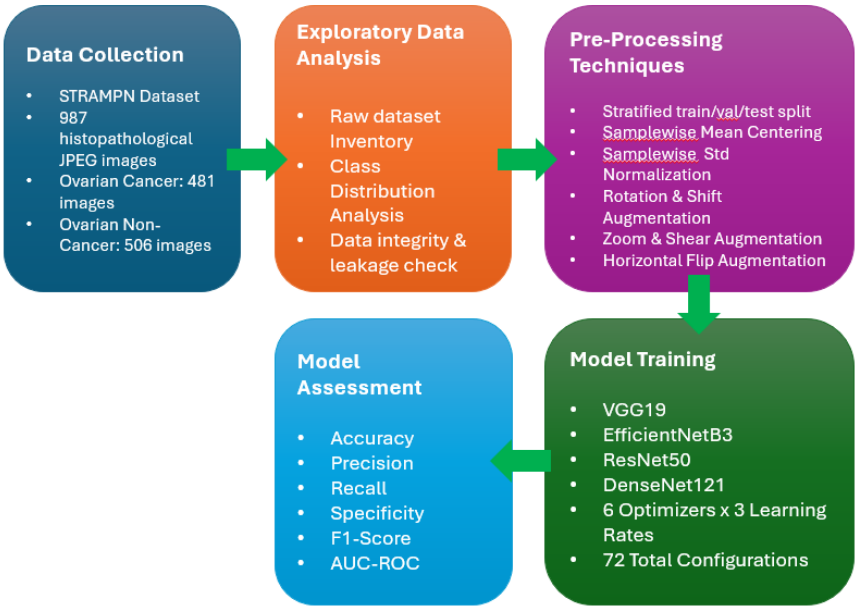


Figure 1: Our experimental workflow. We used a unified data pipeline for all 72 runs to ensure that any differences in performance were due to the model settings, not the data preparation.

By using this structured approach, we were able to systematically compare how each architecture handled different optimizers. This setup also made it easier to identify subopotimal runs early, as we could see when a specific combination of learning rate and optimizer failed to converge regardless of the base model.

5 Methodology

5.1 Experiment Setup

The simulation was built around a full factorial design to see how different combinations of models and optimizers perform. We tested 4 base architectures, 6 optimizers, and 3 learning rates, which gave us a total of $4 \times 6 \times 3 = \mathbf{72}$ unique runs. Each run followed the same training process on our fixed data split to ensure a fair comparison.

We automated the entire process using a master script (`src/run_all.py`) that iterated through every configuration. To keep things organized, each run was saved in its own folder named after the architecture, optimizer, and learning rate (e.g., `VGG19_Adam_LR1e-04/`).

5.2 Architecture and Model Design

We used four convolutional neural networks as our base models: VGG19, EfficientNetB3, ResNet50, and DenseNet121. Following standard transfer learning practices, we loaded these with ImageNet weights and froze all their convolutional layers. This allowed us to use the powerful features these models already learned from millions of images, while we only trained a new classification head on top.

The design of the classification head (Table 4) was provided in the project specifications. It uses a series of Dense layers with Batch Normalization and Dropout to prevent overfitting on our relatively small dataset. We used GlobalAveragePooling2D to bridge the gap between the frozen base and our dense layers, a pattern we adapted from our course materials.

Table 4: Our standard classification head (used for all 72 runs).

Layer	Parameters	Activation
GlobalAveragePooling2D	–	–
Dense	256 units	ReLU
BatchNormalization	–	–
Dropout	0.6 rate	–
Dense	128 units	ReLU
BatchNormalization	–	–
Dropout	0.4 rate	–
Dense	64 units	ReLU
BatchNormalization	–	–
Dropout	0.3 rate	–
Dense (Output)	1 unit	Sigmoid

5.3 Hyperparameters and Optimization

The simulation tested six different optimizers: Adam, Adagrad, Adamax, AdaDelta, SGD (with 0.9 momentum), and RMSProp. For each optimizer, we tried three learning rates: 10^{-4}, 10^{-5}, and 10^{-6}.

Our training parameters were also fixed across all runs:

- **Input Resolution:** 256×256
- **Batch Size:** 32
- **Maximum Epochs:** 50
- **Loss Function:** Binary Crossentropy

To make training efficient, we used several callbacks. We set up `EarlyStopping` to halt runs that stopped improving after 10 epochs, `ReduceLROnPlateau` to adjust the learning rate if the validation loss stalled, and `ModelCheckpoint` to keep only the best weights from each run.

5.4 Metrics and Evaluation

After each run, we evaluated the model on our held-out test set (247 images). We collected the specified metrics: accuracy, precision, and recall, with AUC-ROC as our primary way to rank the models.

Since specificity isn't a default metric in Keras, we implemented a custom calculation to extract it from the confusion matrix. This was important because, in medical imaging, knowing how well a model identifies healthy (non-cancer) tissue is just as critical as finding the cancer.

Table 5: The full 72-run experiment matrix.

Run	Architecture	Optimizer / LR
1–18	VGG19	6 Optimizers $\times$ 3 LRs
19–36	EfficientNetB3	6 Optimizers $\times$ 3 LRs
37–54	ResNet50	6 Optimizers $\times$ 3 LRs
55–72	DenseNet121	6 Optimizers $\times$ 3 LRs

6 Results

6.1 General Performance

Overall, the 72 runs showed a wide range of results. We observed everything from near-perfect classification to complete model failure. The best performance came from VGG19 using Adam and Adamax at a learning rate of 10^{-4}, both reaching an AUC of 0.9995. On the other end of the spectrum, we had 10 subopotimal runs where the model failed to learn anything useful and just predicted one class for every image.

6.2 Top Performers

When we look at the best model for each architecture (Table 6), we see that every base model is capable of high performance if given the right optimizer. VGG19 and DenseNet121 both hit very high AUC scores with Adam, while ResNet50 and EfficientNetB3 performed best with RMSProp. The full suite of resultes are documented in Appendices Sections 9.4 and 9.5.

Table 6: Top configurations for each architecture (ranked by AUC).

Model	Optimizer	LR	Accuracy	AUC
VGG19	Adam	1e-4	77.33%	0.9995
DenseNet121	Adam	1e-4	96.76%	0.9991
ResNet50	RMSProp	1e-4	95.14%	0.9943
EfficientNetB3	RMSProp	1e-4	84.62%	0.9841

6.3 The Impact of Learning Rate

One of our biggest findings was just how much the learning rate matters for transfer learning. As we dropped the learning rate from 10^{-4} down to 10^{-6}, performance took a massive hit across the board.

At 10^{-4}, most models converged quickly and performed well. However, at 10^{-6}, many models stopped improving after just 11 epochs (due to our EarlyStopping callback) because the updates were too small to make any real progress. This tells us that for these specific frozen models, 10^{-4} is likely the optimal rate.

6.4 Optimizer Comparison

In our tests, RMSProp was the most consistent performer, with the highest average AUC across all models. Adam and Adamax were also very strong, especially at the

10^{-4} learning rate. On the other hand, AdaDelta and SGD struggled significantly on this dataset, especially when combined with smaller learning rates.

6.5 Visualizing the Results

We used heatmaps to visualize how the different architectures responded to the optimizer and learning rate settings. Figures 2 and 3 show that while some models are more flexible, most configurations cluster around a specific ideal zone in the hyperparameter space.

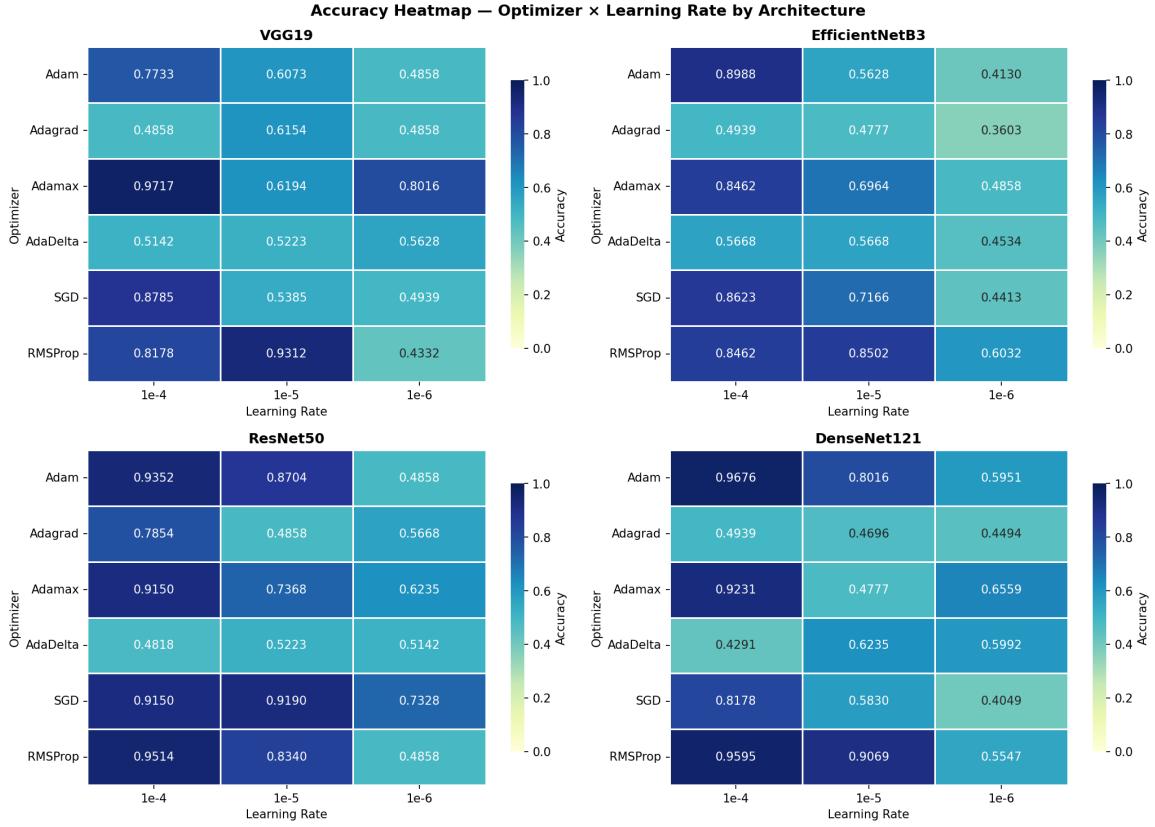


Figure 2: Accuracy across all 72 configurations.

6.6 Failures and Suboptimal Runs

It is also worth noting the runs that failed. We identified 10 cases where the models simply collapsed to predicting one class. Interestingly, some of these runs still showed a high AUC (above 0.80) even though they were practically useless because they never correctly identified a single cancer (or non-cancer) case. This highlighted why we need to look at more than just AUC when evaluating these models for medical use.

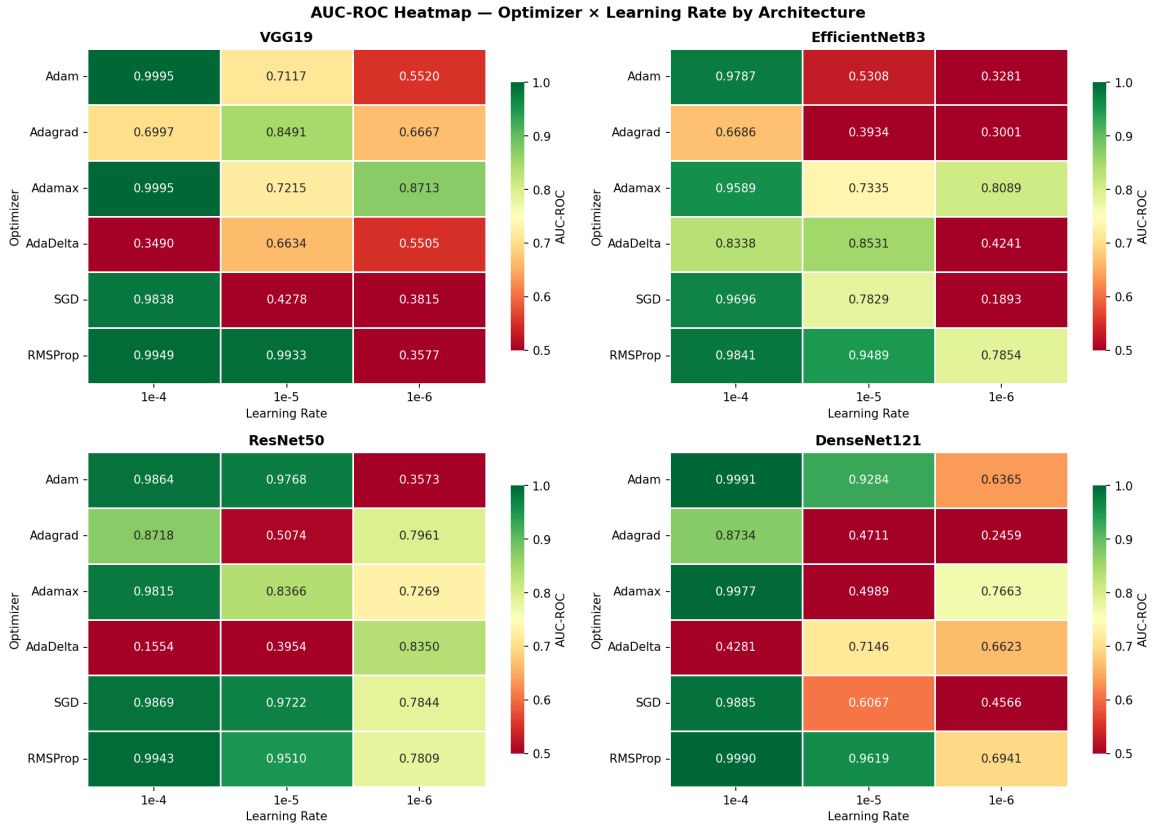


Figure 3: AUC-ROC across all 72 configurations.

7 Discussion

7.1 Why Learning Rate Controlled Everything

The results show a very clear pattern: the learning rate was the single most important factor in whether a model succeeded or failed. At 10^{-6}, almost every model struggled. This makes sense when you consider that we kept the base convolutional layers frozen. Since we were starting the classification head from scratch, those weights needed a decent-sized push to start learning the features extracted by the pre-trained model. At 10^{-6}, the updates were so tiny that the model barely moved from its initial random state before our `EarlyStopping` callback cut it off.

It is worth noting that early stopping any of the configurations does not confirm whether or not the respective model had finished learning. Note that the callback was set with a patience of 10 epochs and a minimum delta of 0.001 on `val_auc`, indicating that the run was halted only when there was no meaningful improvement over 10 consecutive epochs. Within the 72 runs conducted in this study, this limit was set to build consistency amongst the results over seeking out peak performance. Thus, it is worth acknowledging that the results for 1e-6 configurations are representative of the learning rate's performance under the specific conditions of the study as opposed

to the upper limit of what those configuraitons were capable of achieving given more time.

7.2 Comparing the Optimizers

A notable observation was how the different optimizers handled the relatively small dataset. RMSProp was the winner here, followed closely by Adam and Adamax. These adaptive methods are great for medical imaging projects like this because they can adjust their updates based on the gradients they see, which helps when the data is limited and potentially noisy.

In contrast, AdaDelta was surprisingly poor. It is meant to be an improvement over Adagrad, but on the provided dataset, it seemed to stall almost immediately. This may be due to, with so few steps per epoch, AdaDelta's internal learning rate calculation dropping to zero before the model could find a good path.

7.3 Architecture Strengths

ResNet50 was the most reliable performer across the arhitecture suite. This may be due to its skip connections helping keep the gradient signal clean as it passes back to the classification head. VGG19 actually rendered the single best individual run, which might be because its simpler, older architecture produces very straightforward features that the pooling-and-dense head could easily interpret.

EfficientNetB3, which is the most modern model tested, actually came in last on average. Which may be surprising, but it's possible that its more complex feature space needs a lot more data than our 518 training images to really perform to specification.

7.4 A Warning about AUC

One of the most important takeaways for us was about the AUC metric itself. We saw several suboptimal runs where the model predicted the same class for every single image, yet they still had an AUC over 0.80. For instance, the VGG19 + Adam (LR=10^{-6}) (Figure 52c) with gradient updates so miniscule, the model weights never meaningully left from start; leading to all 247 test images to be predicted as cancer without exception, an F1 of 0, and a test accuracy of 48.58% (seemingly random chance) despite a retention of AUC 0.5520. This happens because AUC measures how well the model ranks images, not how well it classifies them at a specific threshold. In a real clinical setting, a model that never finds a single cancer case is a total failure, even if its ranking ability looks good on paper. This reinforces the need to always check recall and specificity alongside AUC.

7.5 Recommendations

Should the research be continued, the next logical step would be fine-tuning. We kept the base models frozen to see what a clean transfer learning performance looked like, but unfreezing the last few layers and training them at a very low learning rate would likely boost the scores significantly. It would be prudent to try k-fold cross-validation to get a better sense of how these models generalize across different subsets of the data. Furthermore, future efforts are recommended to conduct follow-up simulations with a higher early stopping threshold (e.g. 20 or 30 more epochs), or no early stopping at all. The current simulation setup would not be able to meaningfully distinguish between a configuration that failed to learn anything, and a configuration that was terminated before it was given any meaningful time to learn. Isolating learning rate would further clarify whether the learning rate is reflective of the true upper limits of configuration performance or whether it is an artifact of the stopping criteria.

8 Conclusion

In this study, we systematically tested 72 different deep learning configurations to see how they performed at classifying ovarian cancer images. By looking at four different architectures across six optimizers and three learning rates, we have gained a much clearer picture of what works for this kind of medical imaging task.

Our main findings are as follows:

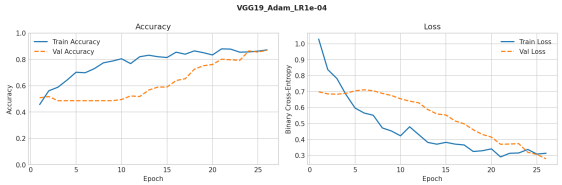
- **Learning Rate is Key:** For frozen transfer learning on this dataset, a learning rate of 10^{-4} is essential. Anything smaller often leads to model collapse.
- **Top Optimizers:** RMSProp, Adam, and Adamax are the most reliable choices for this task.
- **Reliable Models:** ResNet50 was the most consistent overall, while VGG19 produced the single best individual result.
- **Metric Caution:** We can't rely on AUC alone; checking for suboptimal runs using recall and specificity is critical for medical safety.

While these results are promising, they are just a starting point and could use some further work. The hope is for the findings to provide a useful baseline for others working on similar medical classification problems, helping them choose the right optimization settings more quickly and avoid common pitfalls like suboptimal convergence.

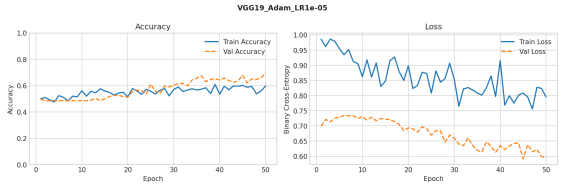
9 Appendices

9.1 Per-Model Accuracy and Loss Curves

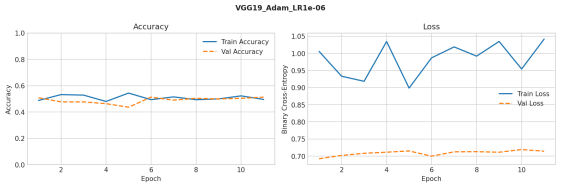
VGG19



(a) LR=1e-4

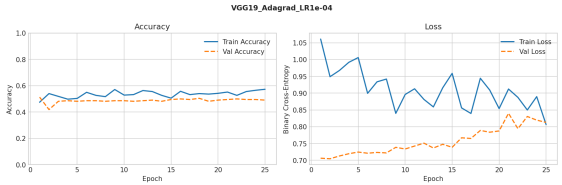


(b) LR=1e-5

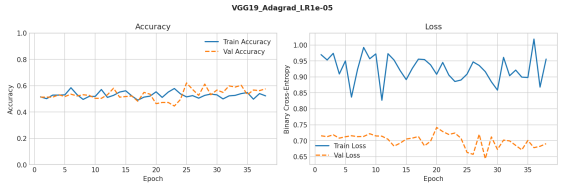


(c) LR=1e-6

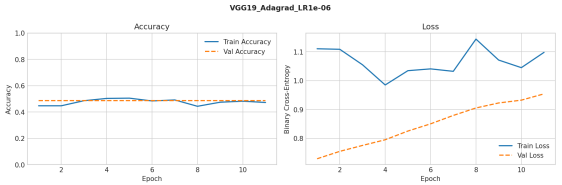
Figure 4: VGG19,Adam: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

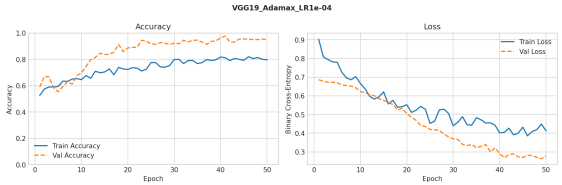


(b) LR=1e-5

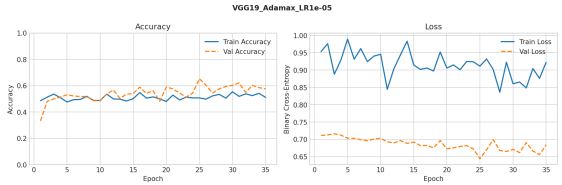


(c) LR=1e-6

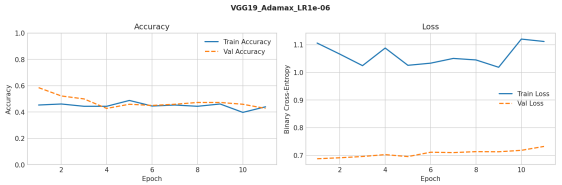
Figure 5: VGG19,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

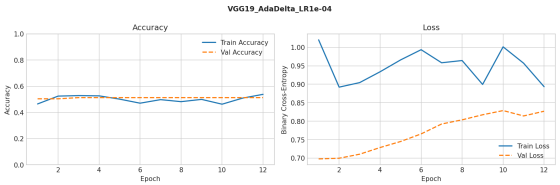


(b) LR=1e-5

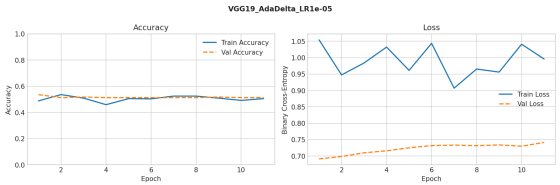


(c) LR=1e-6

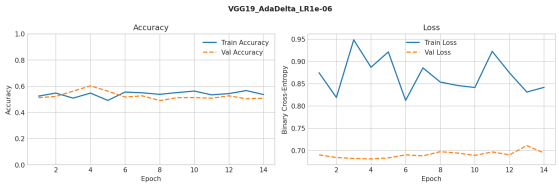
Figure 6: VGG19,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

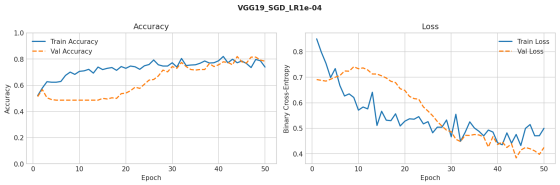


(b) LR=1e-5

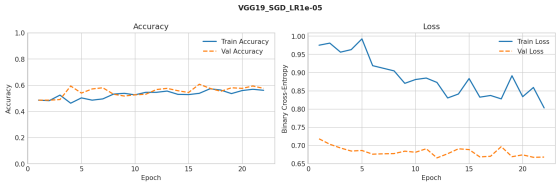


(c) LR=1e-6

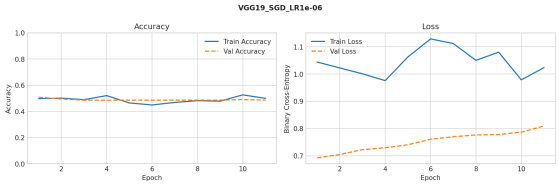
Figure 7: VGG19,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

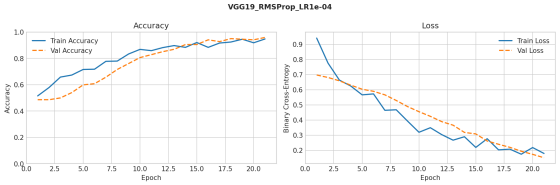


(b) LR=1e-5

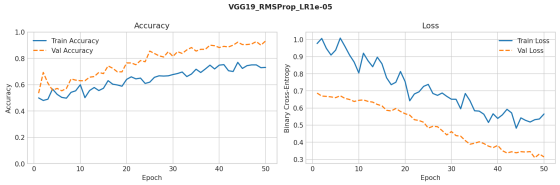


(c) LR=1e-6

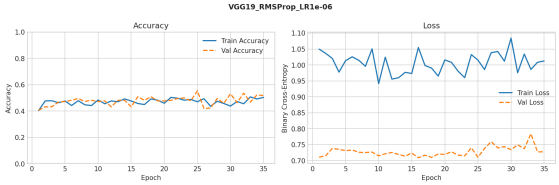
Figure 8: VGG19,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



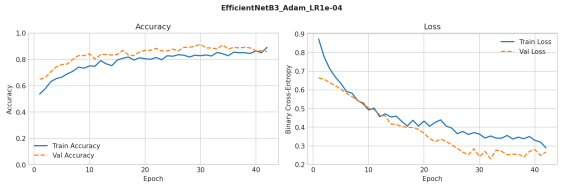
(b) LR=1e-5



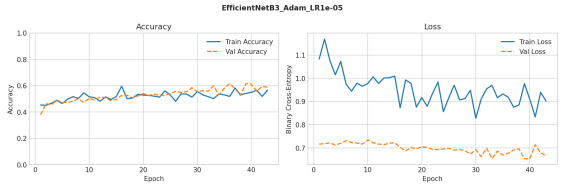
(c) LR=1e-6

Figure 9: VGG19,RMSProp: Accuracy and Loss Curves across Learning Rates

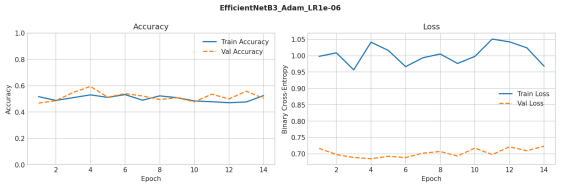
EfficientNetB3



(a) LR=1e-4

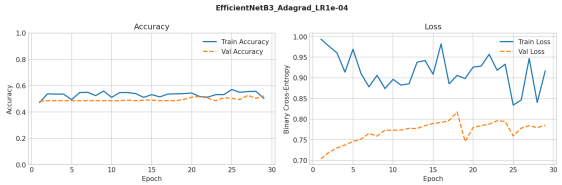


(b) LR=1e-5

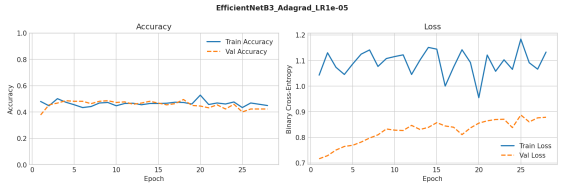


(c) LR=1e-6

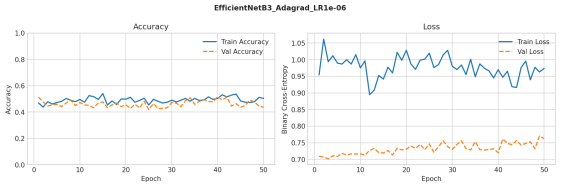
Figure 10: EfficientNetB3,Adam: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

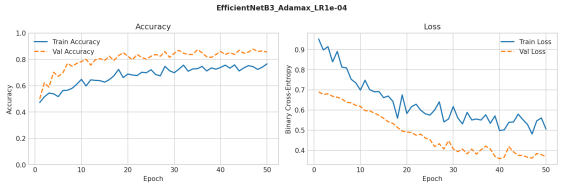


(b) LR=1e-5

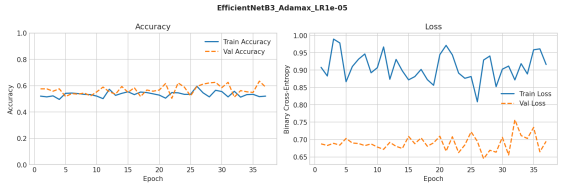


(c) LR=1e-6

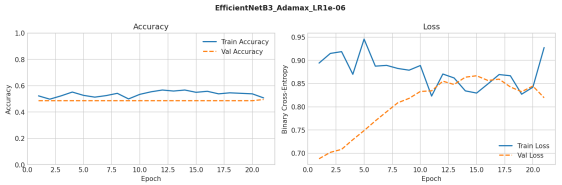
Figure 11: EfficientNetB3,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

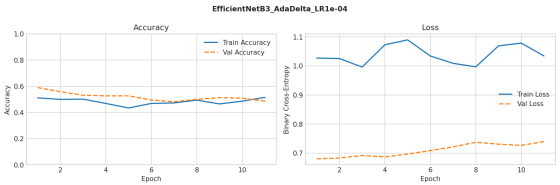


(b) LR=1e-5

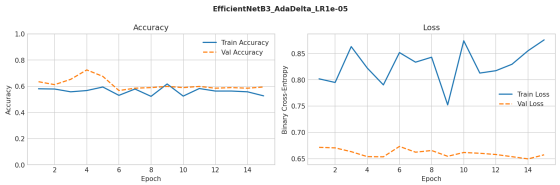


(c) LR=1e-6

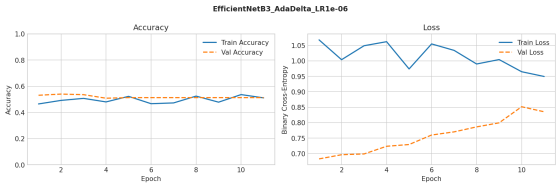
Figure 12: EfficientNetB3,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

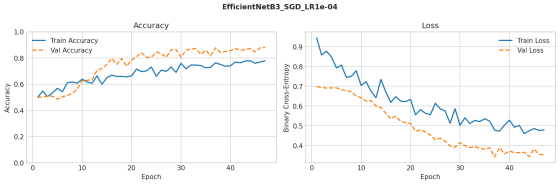


(b) LR=1e-5

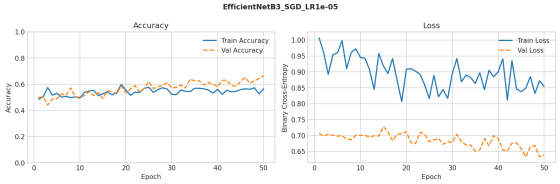


(c) LR=1e-6

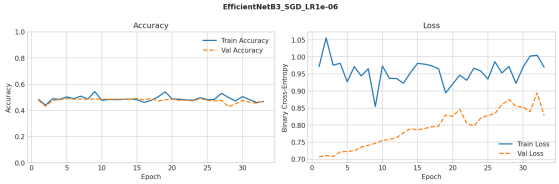
Figure 13: EfficientNetB3,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

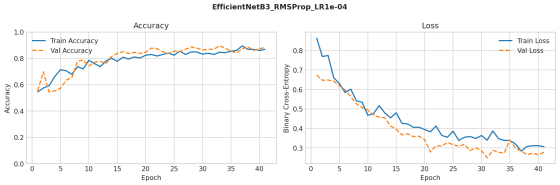


(b) LR=1e-5

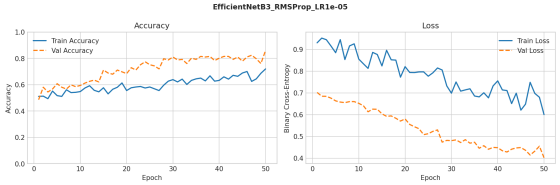


(c) LR=1e-6

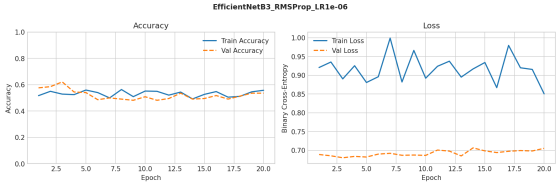
Figure 14: EfficientNetB3,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



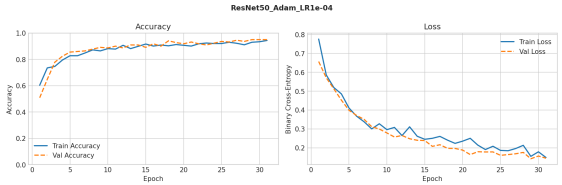
(b) LR=1e-5



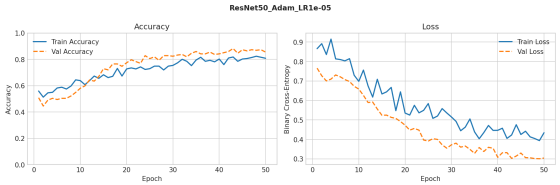
(c) LR=1e-6

Figure 15: EfficientNetB3,RMSProp: Accuracy and Loss Curves across Learning Rates

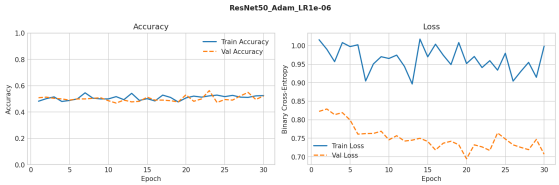
ResNet50



(a) LR=1e-4

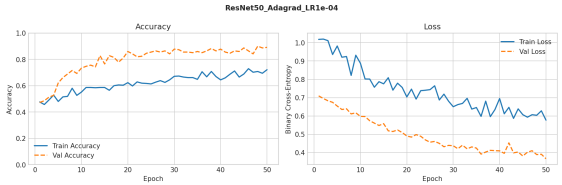


(b) LR=1e-5

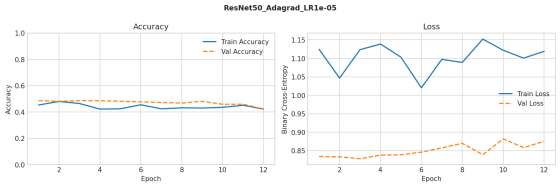


(c) LR=1e-6

Figure 16: ResNet50,Adam: Accuracy and Loss Curves across Learning Rates



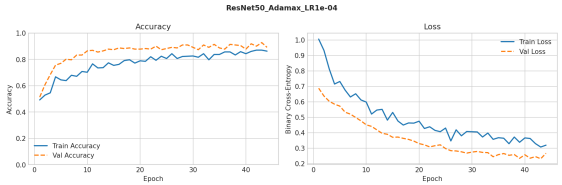
(a) LR=1e-4



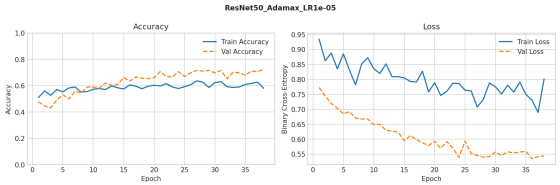
(b) LR=1e-5

(c) LR=1e-6

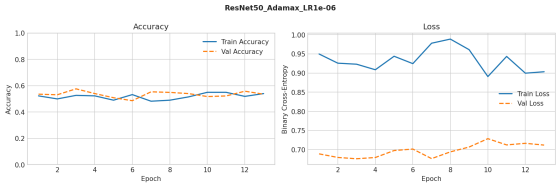
Figure 17: ResNet50,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

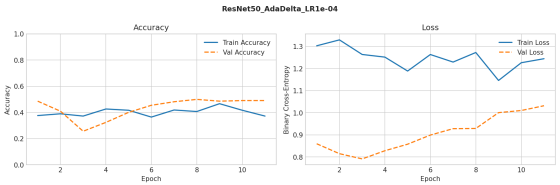


(b) LR=1e-5

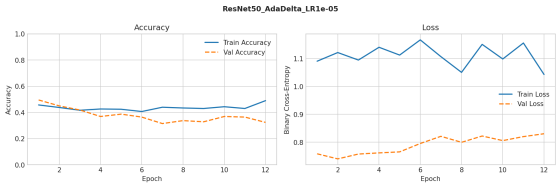


(c) LR=1e-6

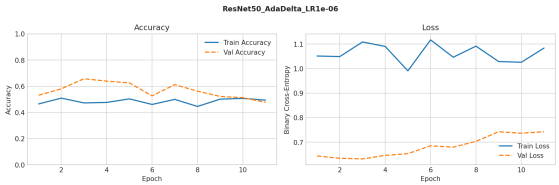
Figure 18: ResNet50,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

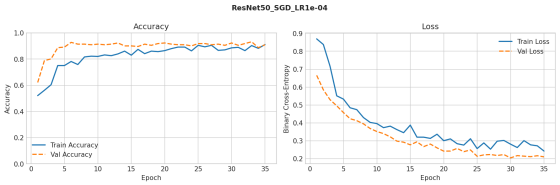


(b) LR=1e-5

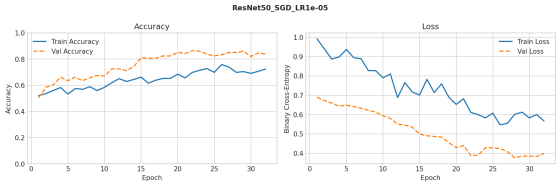


(c) LR=1e-6

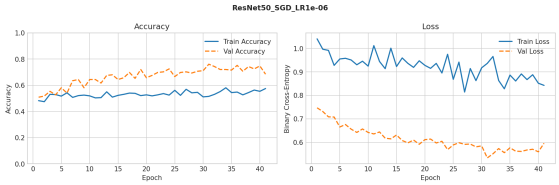
Figure 19: ResNet50,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

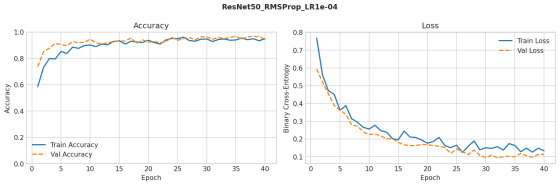


(b) LR=1e-5

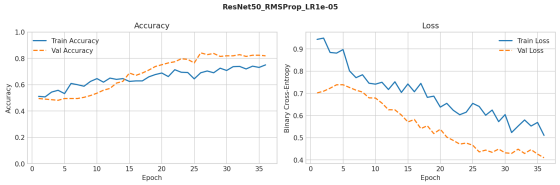


(c) LR=1e-6

Figure 20: ResNet50,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

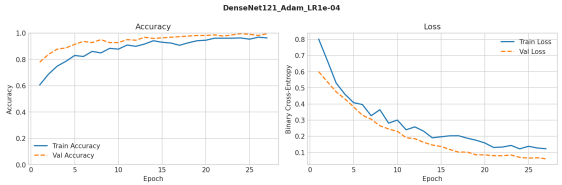


(b) LR=1e-5

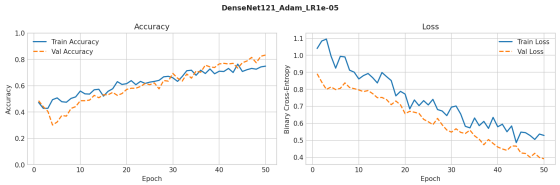
(c) LR=1e-6

Figure 21: ResNet50,RMSProp: Accuracy and Loss Curves across Learning Rates

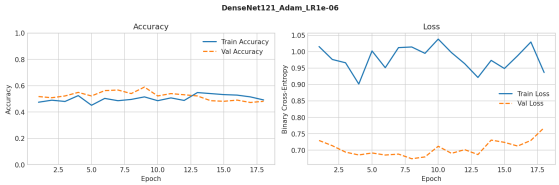
DenseNet121



(a) LR=1e-4

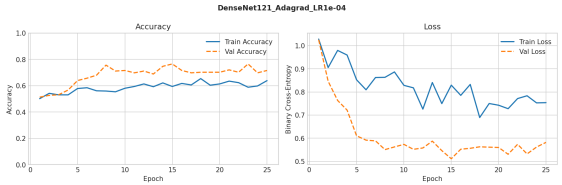


(b) LR=1e-5

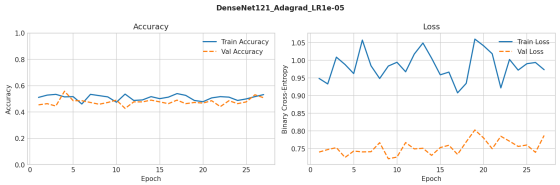


(c) LR=1e-6

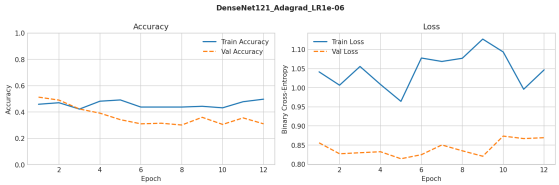
Figure 22: DenseNet121,Adam: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

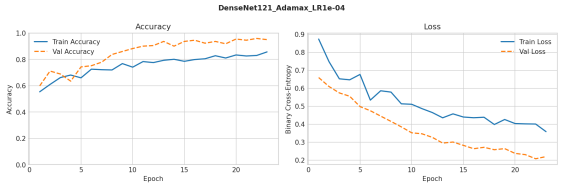


(b) LR=1e-5

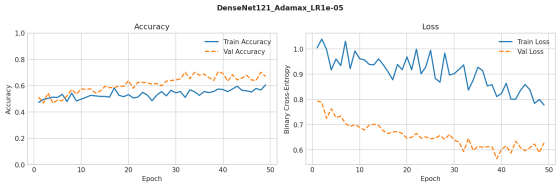


(c) LR=1e-6

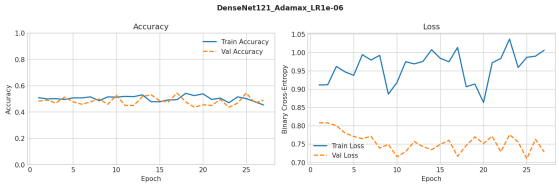
Figure 23: DenseNet121,Adagrad: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

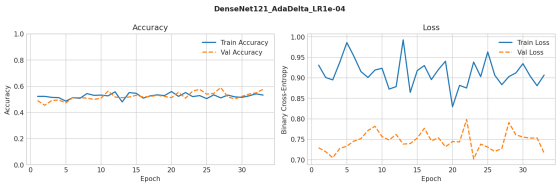


(b) LR=1e-5

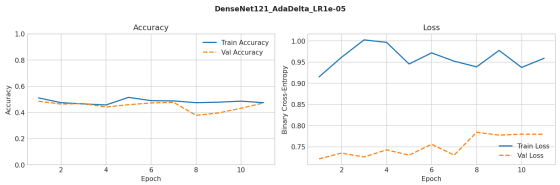


(c) LR=1e-6

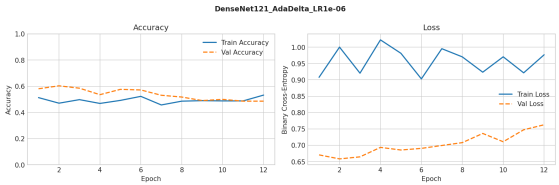
Figure 24: DenseNet121,Adamax: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

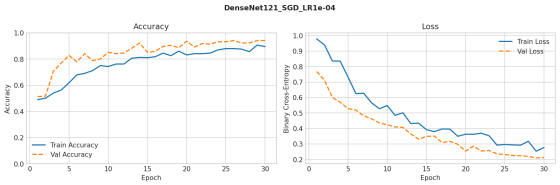


(b) LR=1e-5

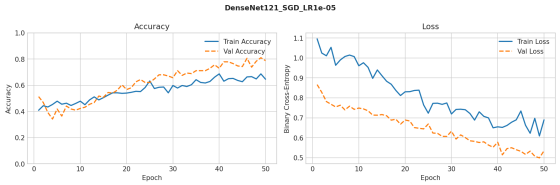


(c) LR=1e-6

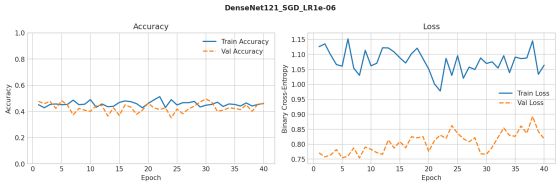
Figure 25: DenseNet121,AdaDelta: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4

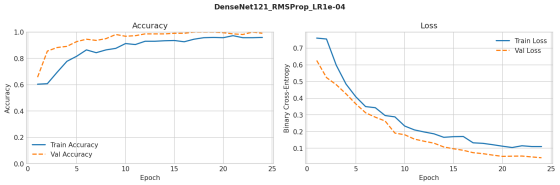


(b) LR=1e-5

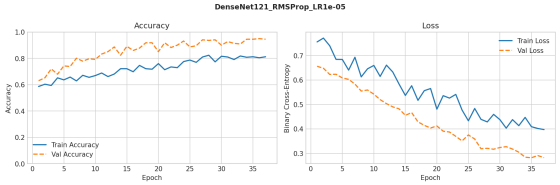


(c) LR=1e-6

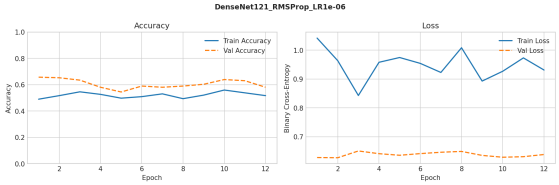
Figure 26: DenseNet121,SGD: Accuracy and Loss Curves across Learning Rates



(a) LR=1e-4



(b) LR=1e-5

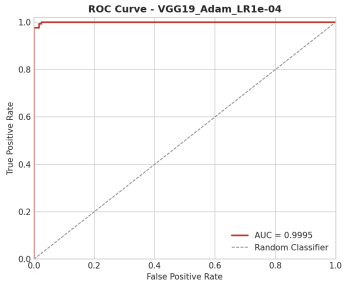


(c) LR=1e-6

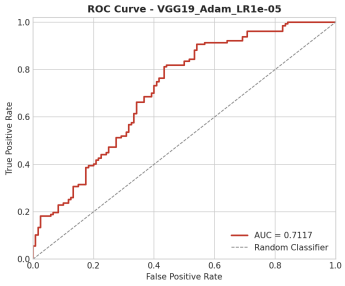
Figure 27: DenseNet121,RMSProp: Accuracy and Loss Curves across Learning Rates

9.2 Per-Model AUC-ROC Curves

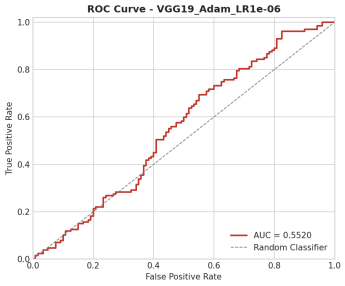
VGG19



(a) LR=1e-4

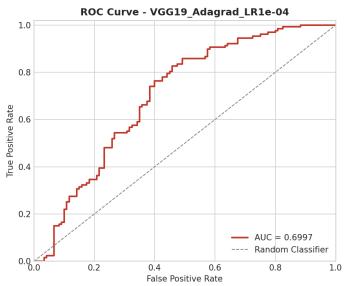


(b) LR=1e-5

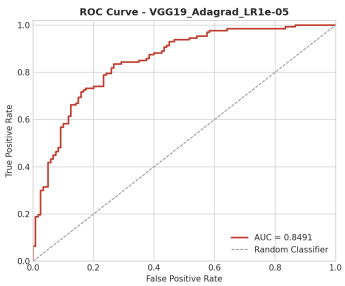


(c) LR=1e-6

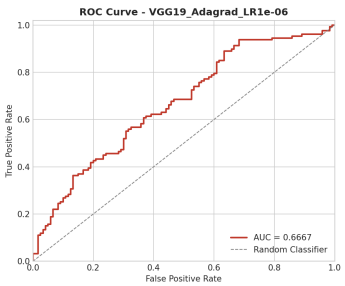
Figure 28: VGG19,Adam: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

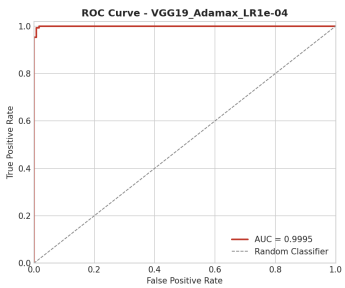


(b) LR=1e-5

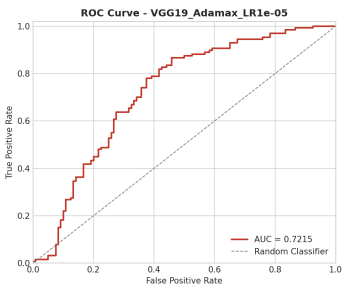


(c) LR=1e-6

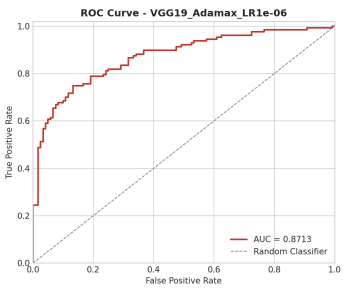
Figure 29: VGG19,Adagrad: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

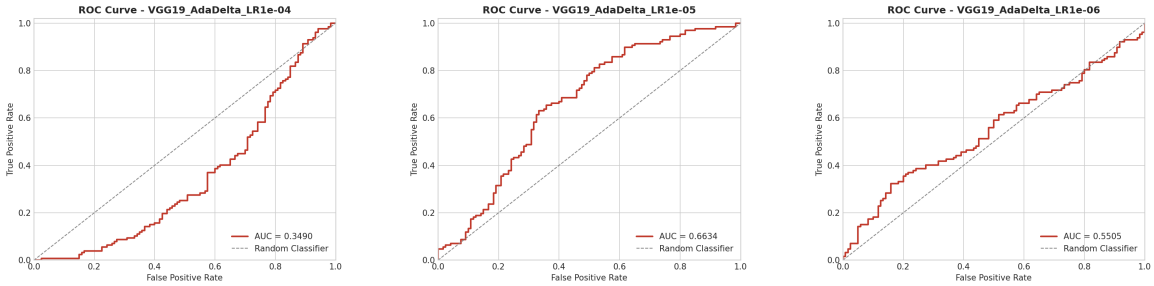


(b) LR=1e-5



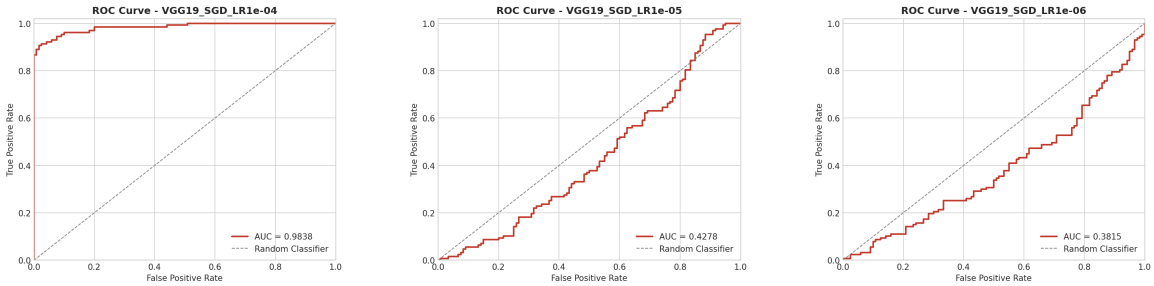
(c) LR=1e-6

Figure 30: VGG19,Adamax: AUC-ROC Curves across Learning Rates



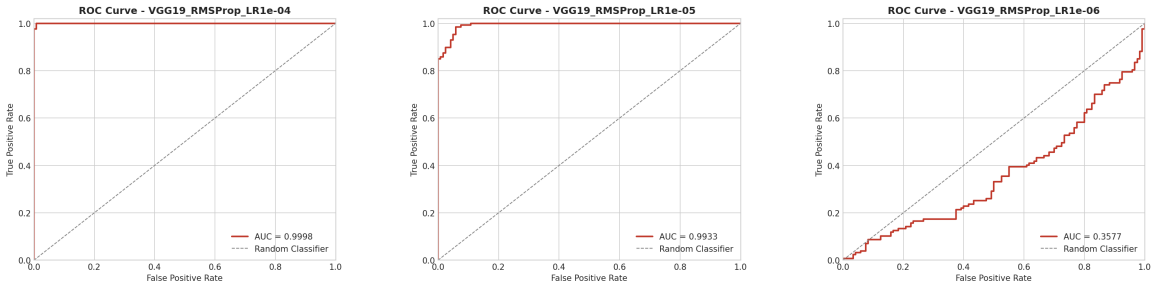
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 31: VGG19,AdaDelta: AUC-ROC Curves across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

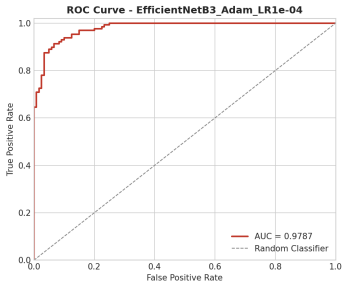
Figure 32: VGG19,SGD: AUC-ROC Curves across Learning Rates



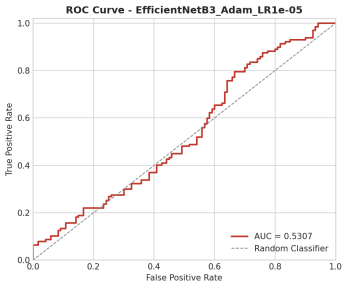
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 33: VGG19,RMSProp: AUC-ROC Curves across Learning Rates

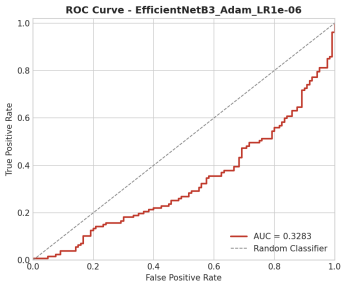
EfficientNetB3



(a) LR=1e-4

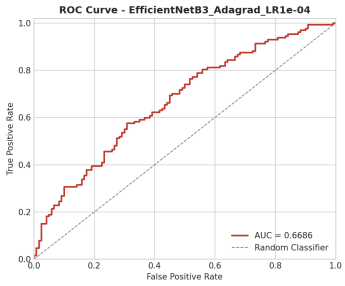


(b) LR=1e-5

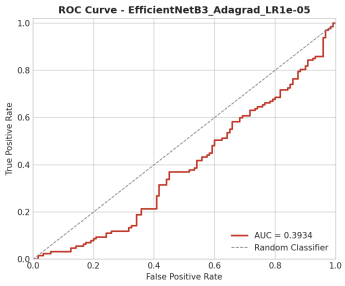


(c) LR=1e-6

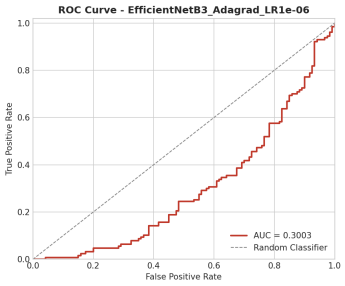
Figure 34: EfficientNetB3,Adam: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

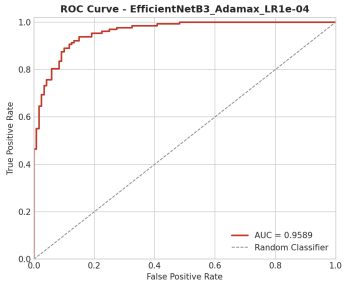


(b) LR=1e-5

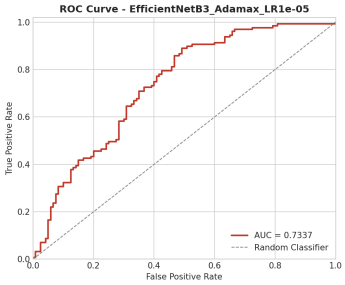


(c) LR=1e-6

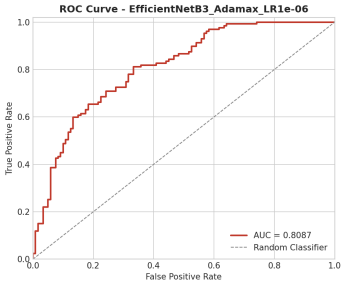
Figure 35: EfficientNetB3,Adagrad: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

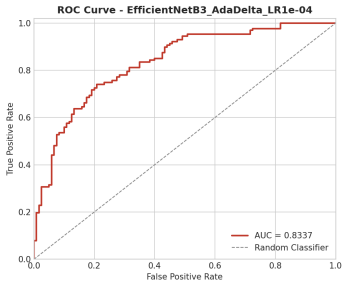


(b) LR=1e-5

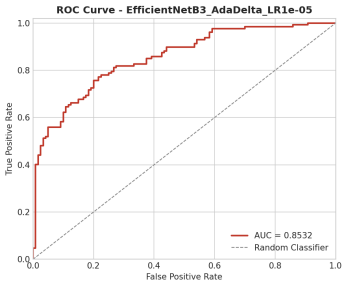


(c) LR=1e-6

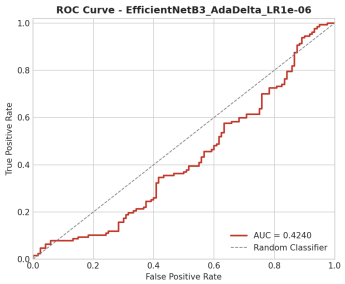
Figure 36: EfficientNetB3,Adamax: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

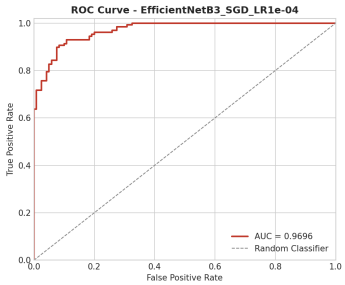


(b) LR=1e-5

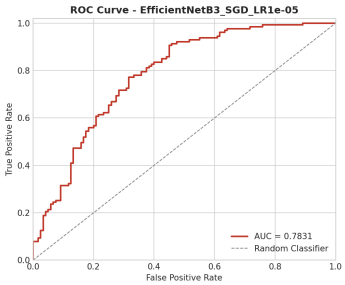


(c) LR=1e-6

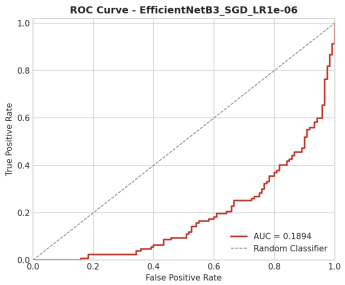
Figure 37: EfficientNetB3,AdaDelta: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

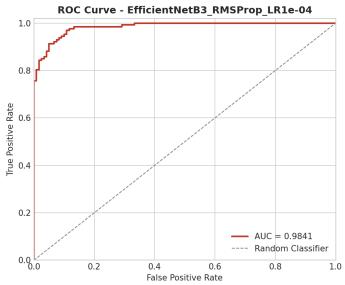


(b) LR=1e-5

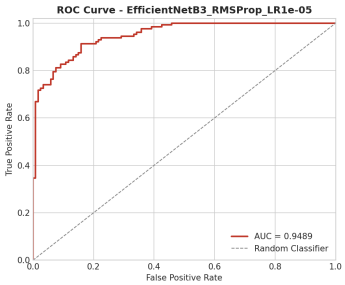


(c) LR=1e-6

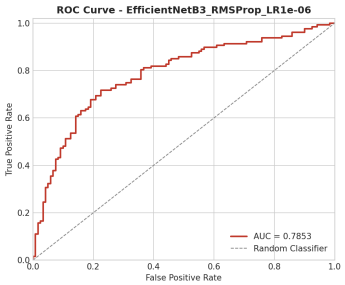
Figure 38: EfficientNetB3,SGD: AUC-ROC Curves across Learning Rates



(a) LR=1e-4



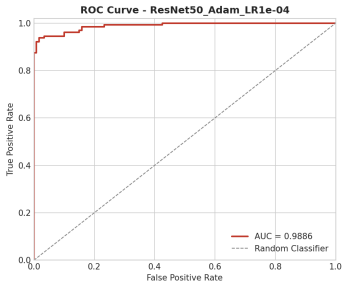
(b) LR=1e-5



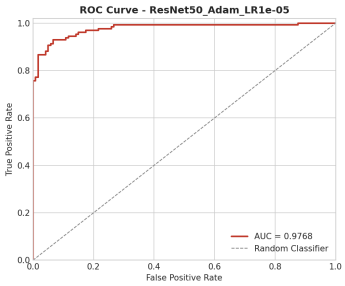
(c) LR=1e-6

Figure 39: EfficientNetB3,RMSProp: AUC-ROC Curves across Learning Rates

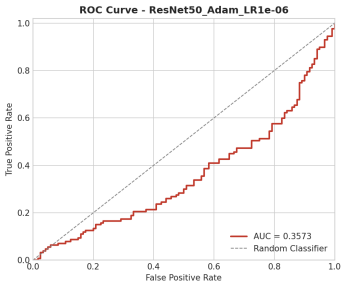
ResNet50



(a) LR=1e-4

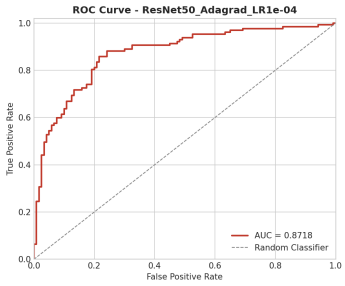


(b) LR=1e-5

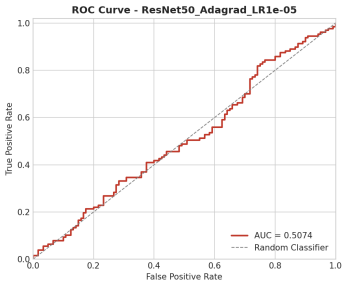


(c) LR=1e-6

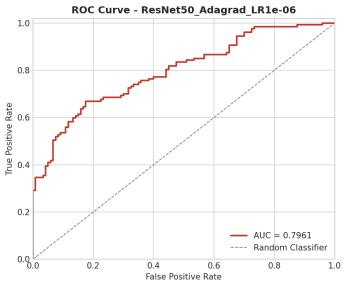
Figure 40: ResNet50,Adam: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

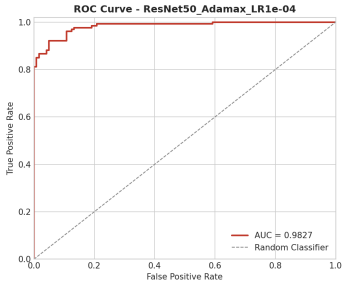


(b) LR=1e-5

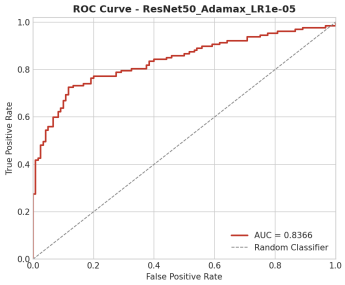


(c) LR=1e-6

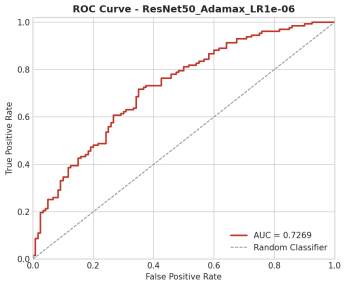
Figure 41: ResNet50,Adagrad: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

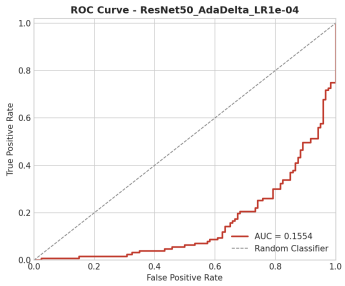


(b) LR=1e-5

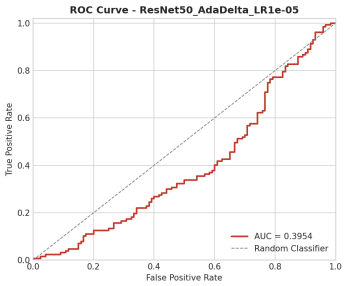


(c) LR=1e-6

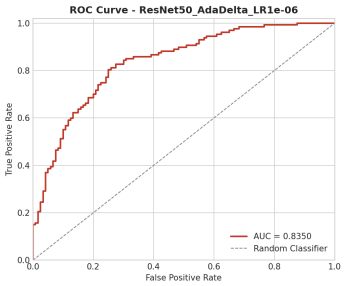
Figure 42: ResNet50,Adamax: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

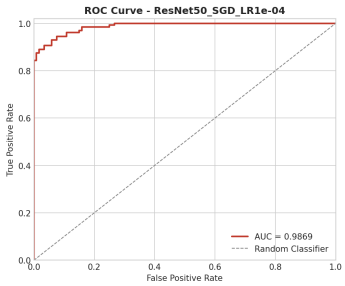


(b) LR=1e-5

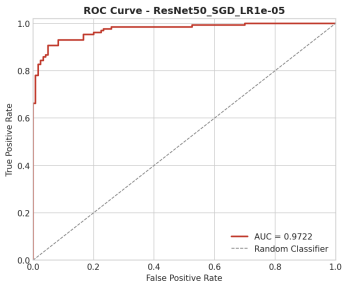


(c) LR=1e-6

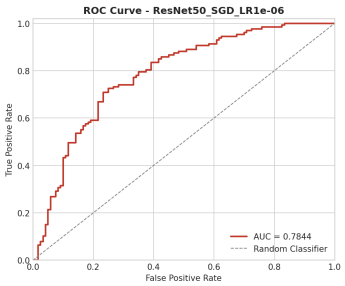
Figure 43: ResNet50,AdaDelta: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

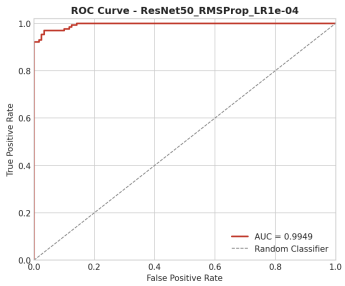


(b) LR=1e-5

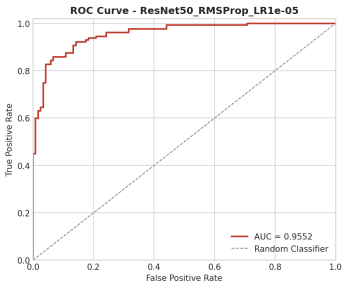


(c) LR=1e-6

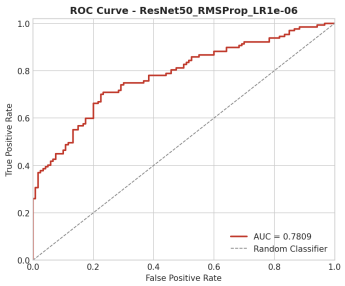
Figure 44: ResNet50,SGD: AUC-ROC Curves across Learning Rates



(a) LR=1e-4



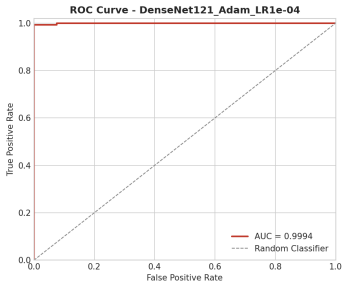
(b) LR=1e-5



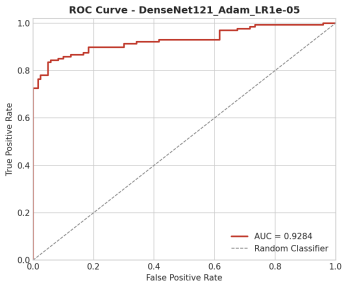
(c) LR=1e-6

Figure 45: ResNet50,RMSProp: AUC-ROC Curves across Learning Rates

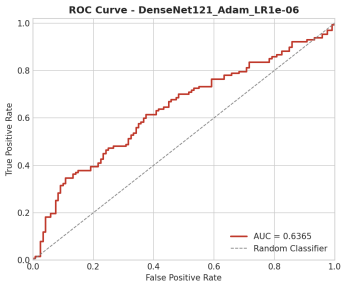
DenseNet121



(a) LR=1e-4

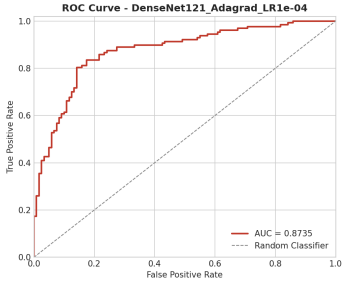


(b) LR=1e-5

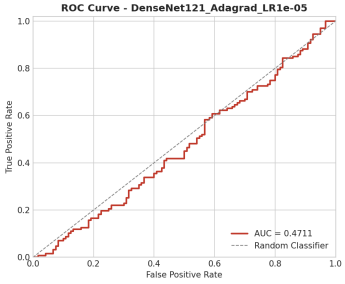


(c) LR=1e-6

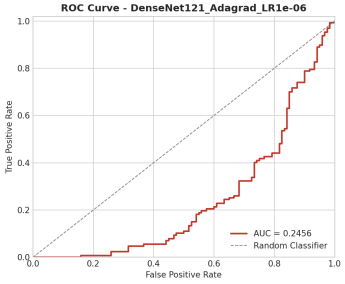
Figure 46: DenseNet121,Adam: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

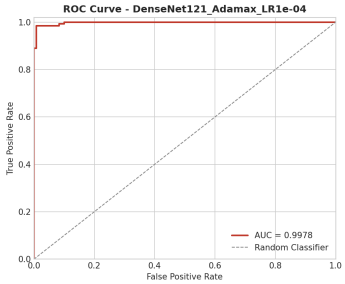


(b) LR=1e-5

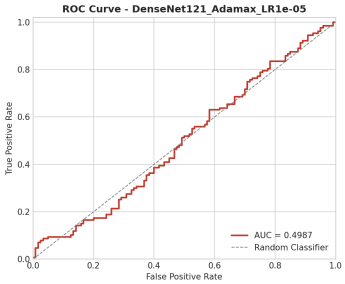


(c) LR=1e-6

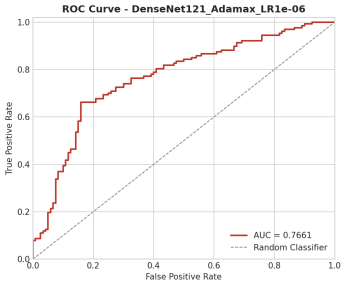
Figure 47: DenseNet121,Adagrad: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

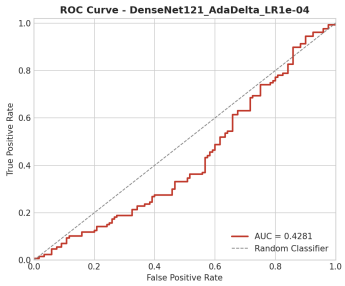


(b) LR=1e-5

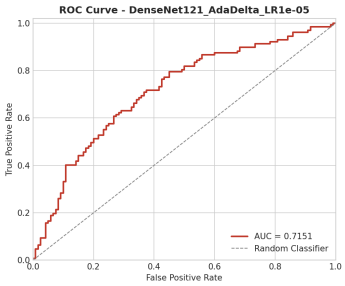


(c) LR=1e-6

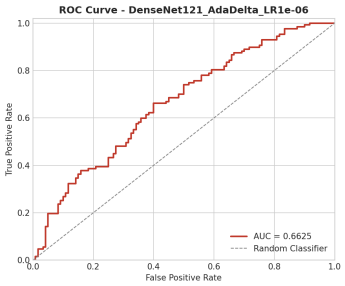
Figure 48: DenseNet121,Adamax: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

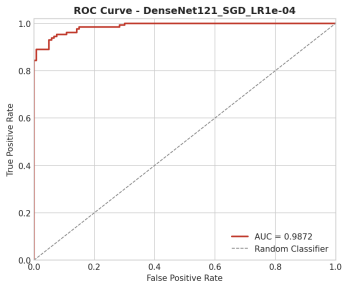


(b) LR=1e-5

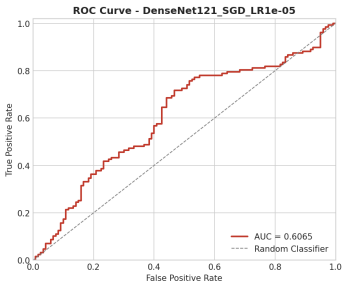


(c) LR=1e-6

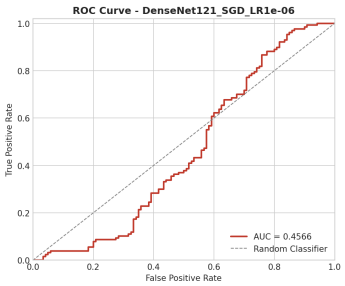
Figure 49: DenseNet121,AdaDelta: AUC-ROC Curves across Learning Rates



(a) LR=1e-4

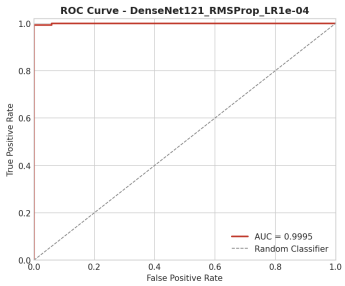


(b) LR=1e-5

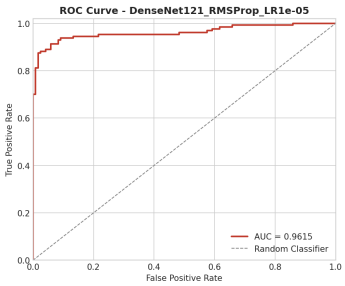


(c) LR=1e-6

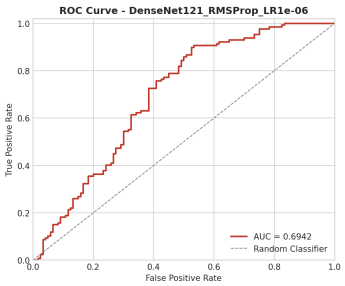
Figure 50: DenseNet121,SGD: AUC-ROC Curves across Learning Rates



(a) LR=1e-4



(b) LR=1e-5

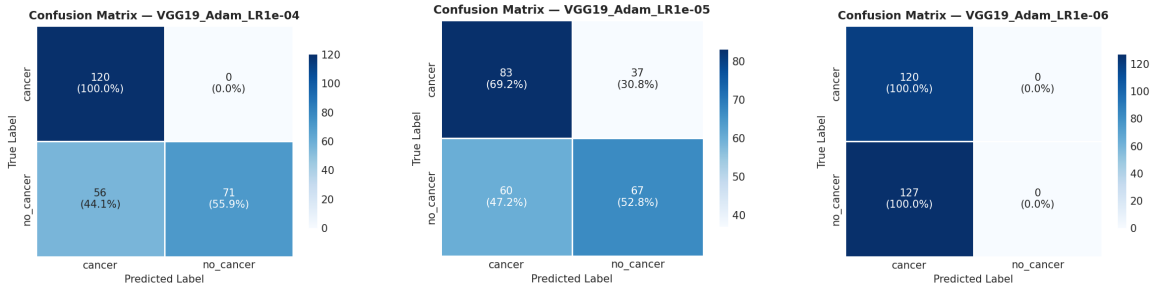


(c) LR=1e-6

Figure 51: DenseNet121,RMSProp: AUC-ROC Curves across Learning Rates

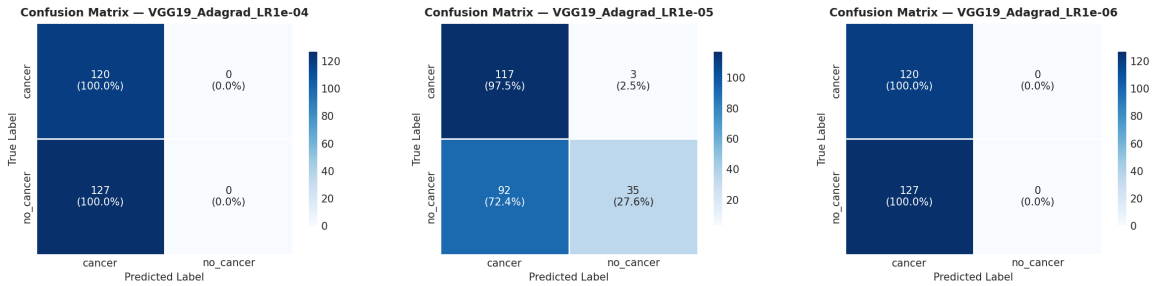
9.3 Per-Model Confusion Matrices

VGG19



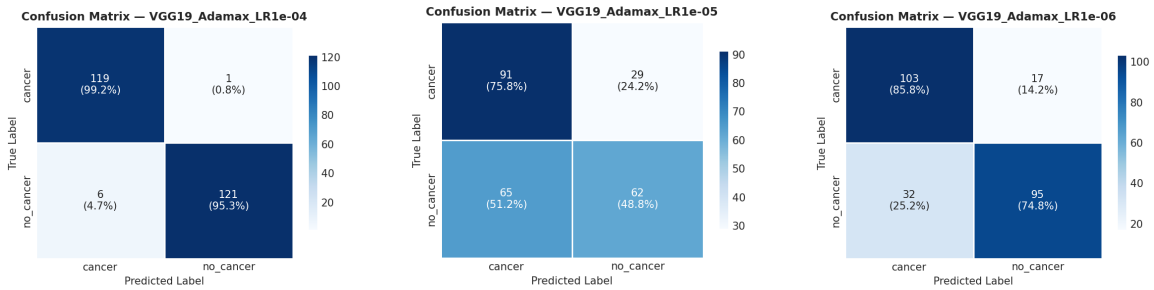
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 52: VGG19,Adam: Confusion Matrices across Learning Rates



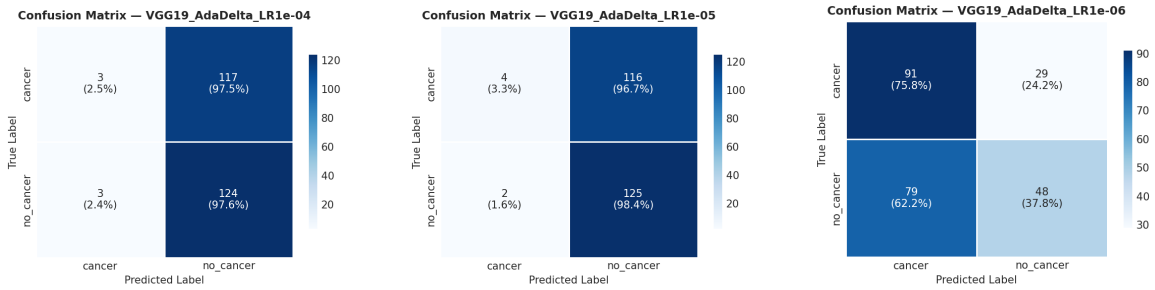
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 53: VGG19,Adagrad: Confusion Matrices across Learning Rates



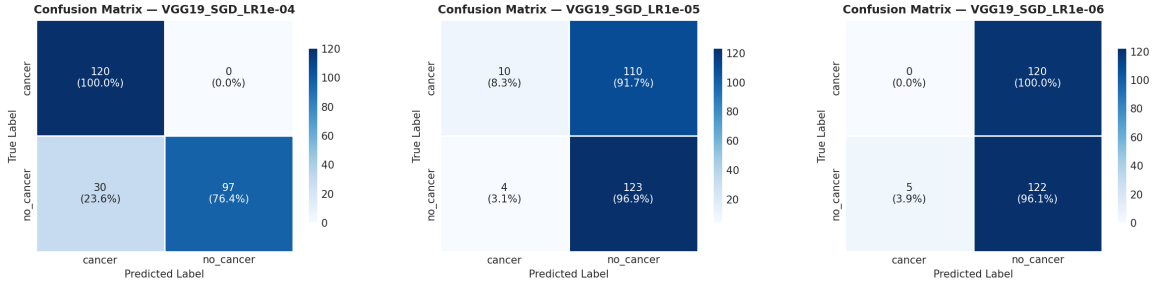
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 54: VGG19,Adamax: Confusion Matrices across Learning Rates



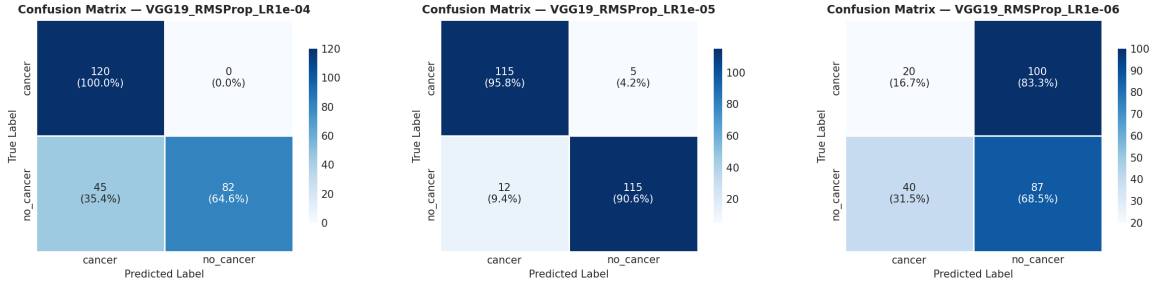
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 55: VGG19,AdaDelta: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

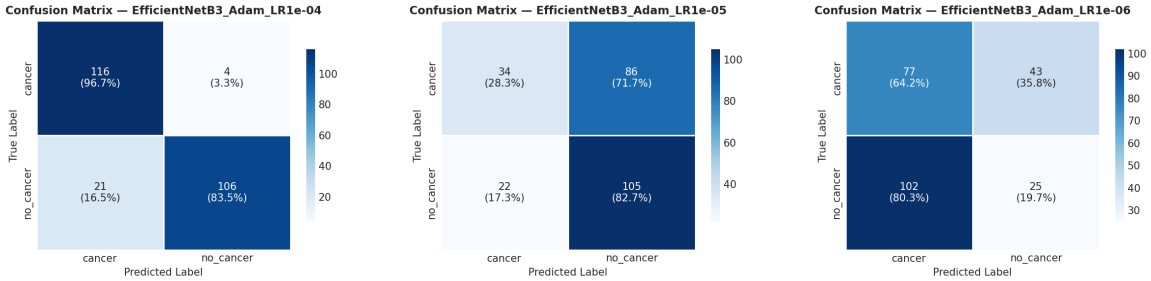
Figure 56: VGG19,SGD: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

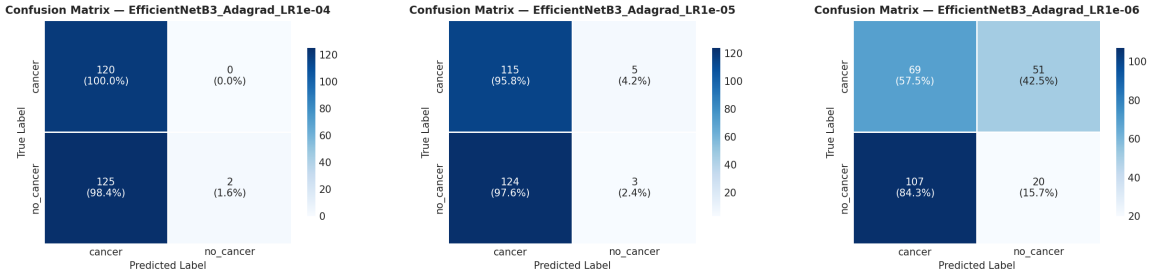
Figure 57: VGG19,RMSProp: Confusion Matrices across Learning Rates

EfficientNetB3



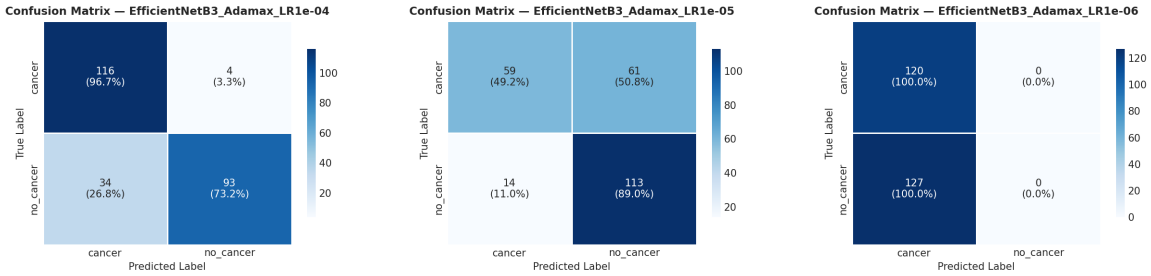
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 58: EfficientNetB3,Adam: Confusion Matrices across Learning Rates



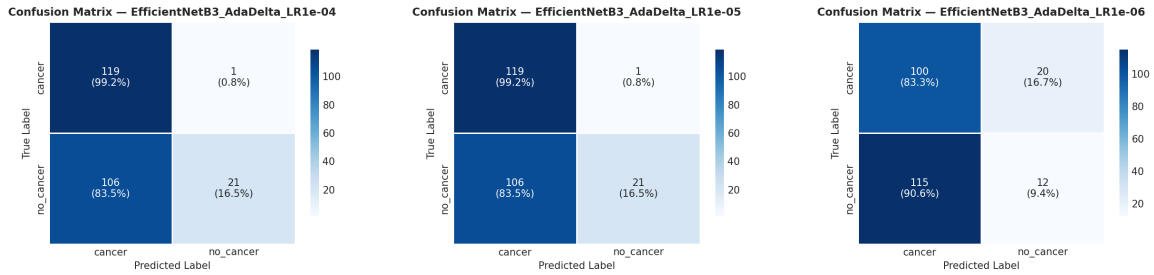
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 59: EfficientNetB3,Adagrad: Confusion Matrices across Learning Rates



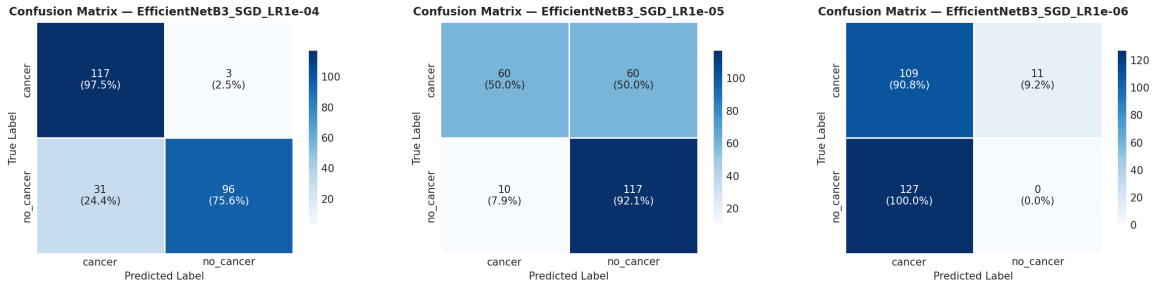
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 60: EfficientNetB3,Adamax: Confusion Matrices across Learning Rates



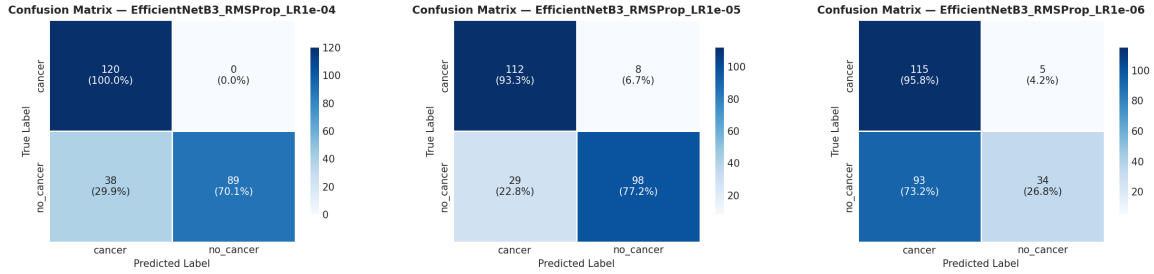
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 61: EfficientNetB3,AdaDelta: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

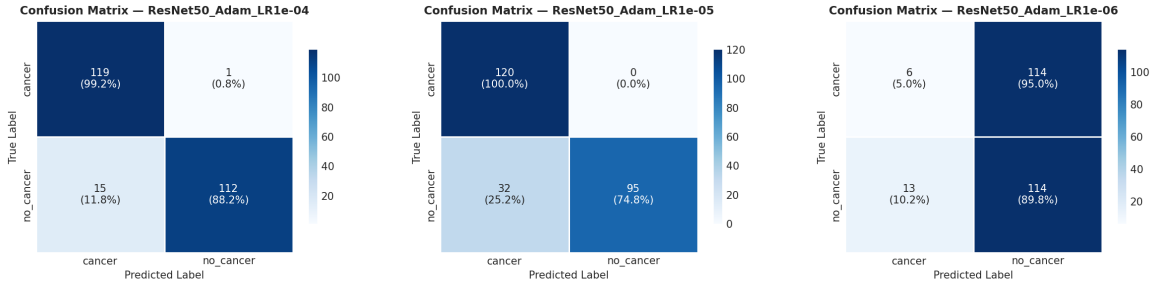
Figure 62: EfficientNetB3,SGD: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

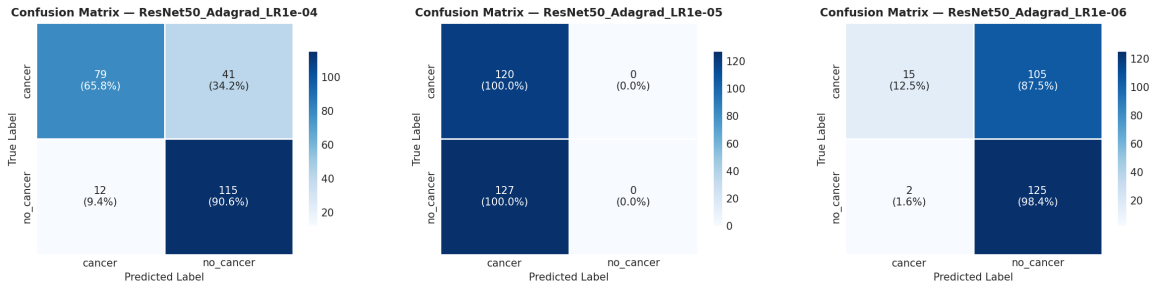
Figure 63: EfficientNetB3,RMSProp: Confusion Matrices across Learning Rates

ResNet50



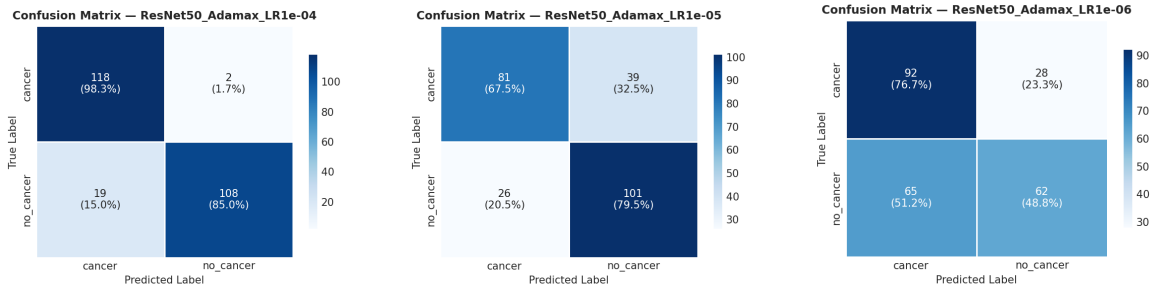
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 64: ResNet50,Adam: Confusion Matrices across Learning Rates



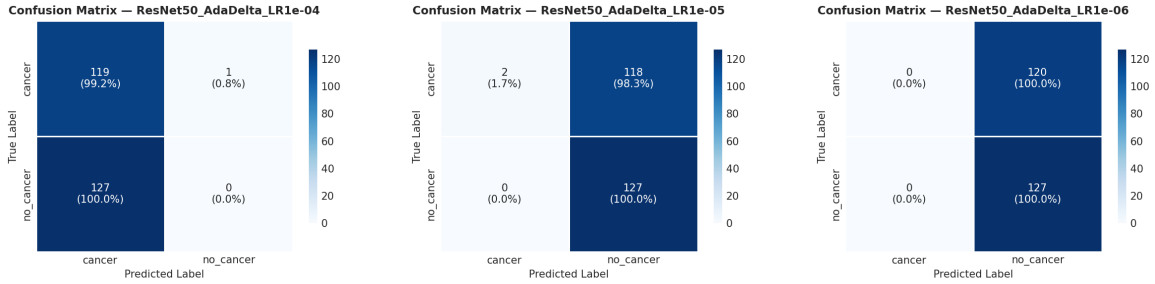
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 65: ResNet50,Adagrad: Confusion Matrices across Learning Rates



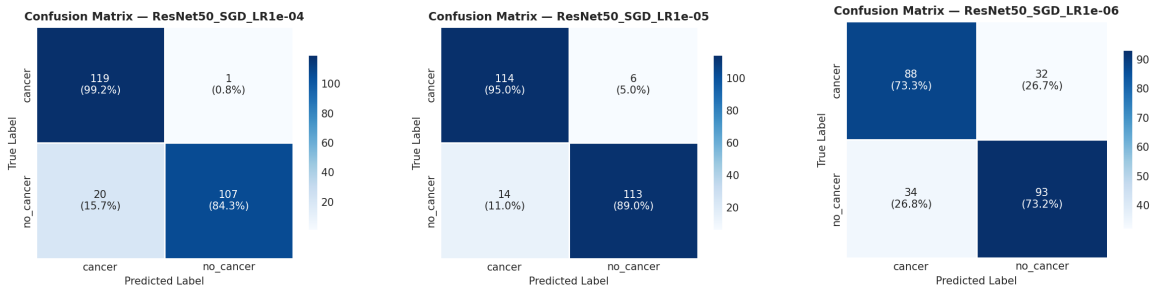
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 66: ResNet50,Adamax: Confusion Matrices across Learning Rates



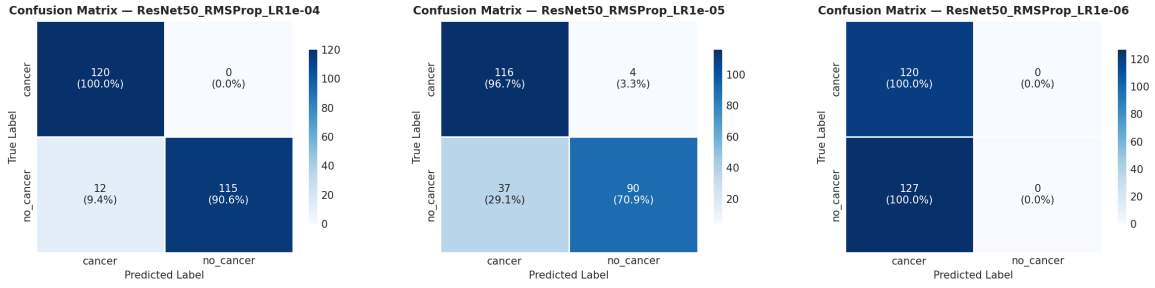
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 67: ResNet50,AdaDelta: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

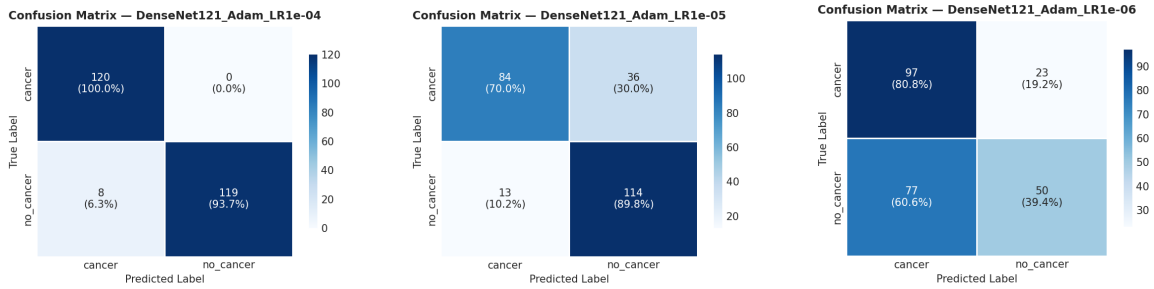
Figure 68: ResNet50,SGD: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

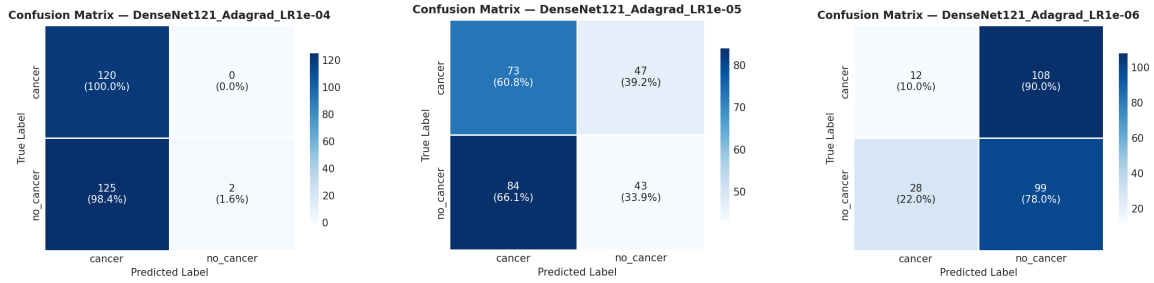
Figure 69: ResNet50,RMSProp: Confusion Matrices across Learning Rates

DenseNet121



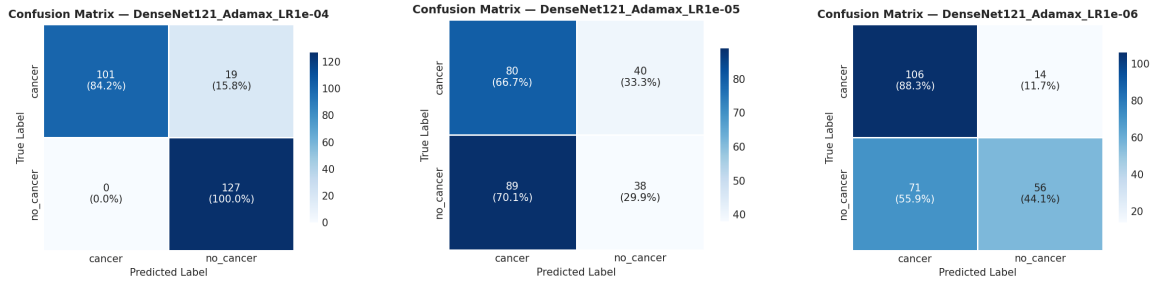
(a) LR=1e-4 (b) LR=1e-5 (c) LR=1e-6

Figure 70: DenseNet121,Adam: Confusion Matrices across Learning Rates



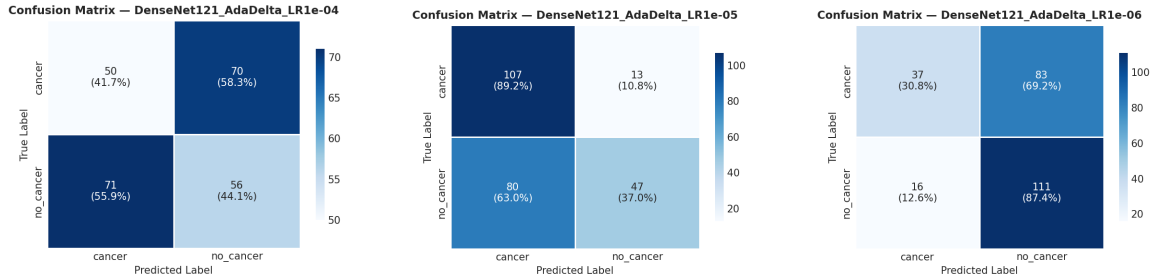
(a) LR=1e-4 (b) LR=1e-5 (c) LR=1e-6

Figure 71: DenseNet121,Adagrad: Confusion Matrices across Learning Rates



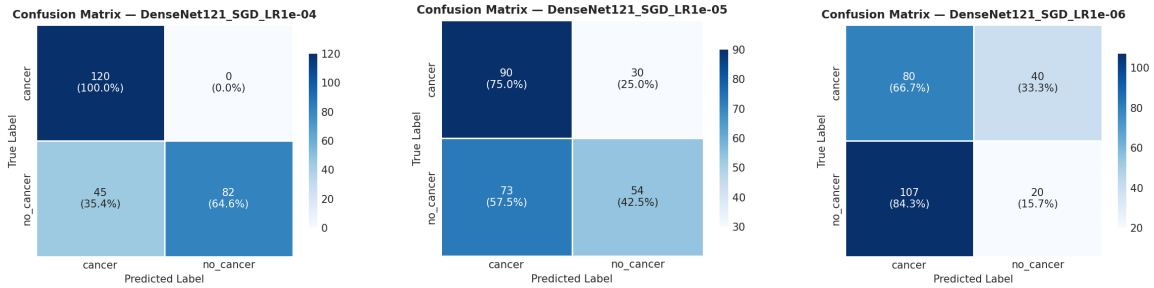
(a) LR=1e-4 (b) LR=1e-5 (c) LR=1e-6

Figure 72: DenseNet121,Adamax: Confusion Matrices across Learning Rates



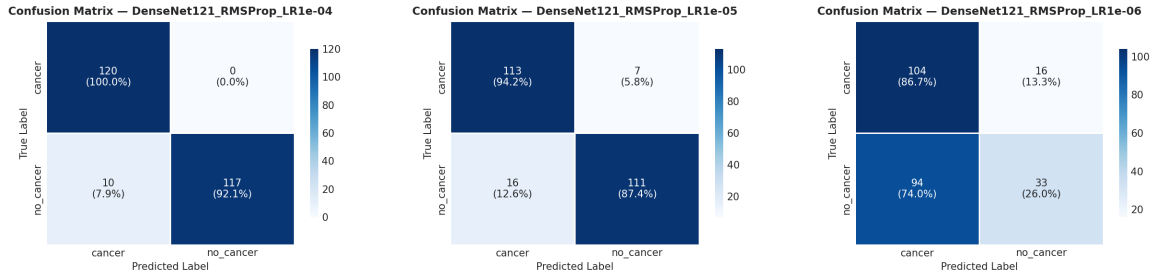
(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 73: DenseNet121,AdaDelta: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 74: DenseNet121,SGD: Confusion Matrices across Learning Rates



(a) LR=1e-4 **(b)** LR=1e-5 **(c)** LR=1e-6

Figure 75: DenseNet121,RMSProp: Confusion Matrices across Learning Rates

9.4 Complete Metric Suite per Architecture

9.4.1 VGG19

Table 7: VGG19 results at learning rate 1e-4

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adam	0.7733	1.0000	0.5591	1.0000	0.7172	0.9995	26
Adamax	**0.9717**	**0.9918**	**0.9528**	**0.9917**	**0.9719**	**0.9995**	**50**
RMSProp	0.8178	1.0000	0.6457	1.0000	0.7847	0.9949	21
SGD	0.8785	1.0000	0.7638	1.0000	0.8661	0.9838	50
Adagrad	0.4858	0.0000	0.0000	1.0000	0.0000	0.6997	25
AdaDelta	0.5142	0.5145	0.9764	0.0250	0.6739	0.3490	12

Learning Rate: 1e-4

Table 8: VGG19 results at learning rate 1e-5

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
RMSProp	**0.9312**	**0.9583**	**0.9055**	**0.9583**	**0.9312**	**0.9933**	**50**
Adagrad	0.6154	0.9211	0.2756	0.9750	0.4242	0.8491	38
Adamax	0.6194	0.6813	0.4882	0.7583	0.5688	0.7215	35
Adam	0.6073	0.6442	0.5276	0.6917	0.5801	0.7117	50
AdaDelta	0.5223	0.5187	0.9843	0.0333	0.6793	0.6634	11
SGD	0.5385	0.5279	0.9685	0.0833	0.6833	0.4278	22

Learning Rate: 1e-5

Table 9: VGG19 results at learning rate 1e-6

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adamax	**0.8016**	**0.8482**	**0.7480**	**0.8583**	**0.7950**	**0.8713**	**11**
Adagrad	0.4858	0.0000	0.0000	1.0000	0.0000	0.6667	11
Adam	0.4858	0.0000	0.0000	1.0000	0.0000	0.5520	11
AdaDelta	0.5628	0.6234	0.3780	0.7583	0.4706	0.5505	14
SGD	0.4939	0.5041	0.9606	0.0000	0.6612	0.3815	11
RMSProp	0.4332	0.4652	0.6850	0.1667	0.5541	0.3577	35

Learning Rate: 1e-6

9.4.2 ResNet50

Table 10: ResNet50 results at learning rate 1e-4

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
RMSProp	**0.9514**	**1.0000**	**0.9055**	**1.0000**	**0.9504**	**0.9943**	**40**
SGD	0.9150	0.9907	0.8425	0.9917	0.9106	0.9869	35
Adam	0.9352	0.9912	0.8819	0.9917	0.9333	0.9864	31
Adamax	0.9150	0.9818	0.8504	0.9833	0.9114	0.9815	44
Adagrad	0.7854	0.7372	0.9055	0.6583	0.8127	0.8718	50
AdaDelta	0.4818	0.0000	0.0000	0.9917	0.0000	0.1554	11

Learning Rate: 1e-4

Table 11: ResNet50 results at learning rate 1e-5

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adam	**0.8704**	**1.0000**	**0.7480**	**1.0000**	**0.8559**	**0.9768**	**50**
SGD	0.9190	0.9496	0.8898	0.9500	0.9187	0.9722	32
RMSProp	0.8340	0.9574	0.7087	0.9667	0.8145	0.9510	36
Adamax	0.7368	0.7214	0.7953	0.6750	0.7566	0.8366	38
Adagrad	0.4858	0.0000	0.0000	1.0000	0.0000	0.5074	12
AdaDelta	0.5223	0.5184	1.0000	0.0167	0.6828	0.3954	12

Learning Rate: 1e-5

Table 12: ResNet50 results at learning rate 1e-6

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
AdaDelta	0.5142	0.5142	1.0000	0.0000	0.6791	0.8350	11
Adagrad	0.5668	0.5435	0.9843	0.1250	0.7003	0.7961	25
SGD	**0.7328**	**0.7440**	**0.7323**	**0.7333**	**0.7381**	**0.7844**	**41**
RMSProp	0.4858	0.0000	0.0000	1.0000	0.0000	0.7809	11
Adamax	0.6235	0.6889	0.4882	0.7667	0.5714	0.7269	13
Adam	0.4858	0.5000	0.8976	0.0500	0.6423	0.3573	30

Learning Rate: 1e-6

9.4.3 DenseNet121

Table 13: DenseNet121 results at learning rate 1e-4

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adam	**0.9676**	**1.0000**	**0.9370**	**1.0000**	**0.9675**	**0.9991**	**27**
RMSProp	0.9595	1.0000	0.9213	1.0000	0.9590	0.9990	24
Adamax	0.9231	0.8699	1.0000	0.8417	0.9304	0.9977	23
SGD	0.8178	1.0000	0.6457	1.0000	0.7847	0.9885	30
Adagrad	0.4939	1.0000	0.0157	1.0000	0.0310	0.8734	25
AdaDelta	0.4291	0.4444	0.4409	0.4167	0.4427	0.4281	33

Learning Rate: 1e-4

Table 14: DenseNet121 results at learning rate 1e-5

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
RMSProp	**0.9069**	**0.9407**	**0.8740**	**0.9417**	**0.9061**	**0.9619**	**37**
Adam	0.8016	0.7600	0.8976	0.7000	0.8231	0.9284	50
AdaDelta	0.6235	0.7833	0.3701	0.8917	0.5027	0.7146	11
SGD	0.5830	0.6429	0.4252	0.7500	0.5118	0.6067	50
Adamax	0.4777	0.4872	0.2992	0.6667	0.3707	0.4989	49
Adagrad	0.4696	0.4778	0.3386	0.6083	0.3963	0.4711	27

Learning Rate: 1e-5

Table 15: DenseNet121 results at learning rate 1e-6

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adamax	**0.6559**	**0.8000**	**0.4409**	**0.8833**	**0.5685**	**0.7663**	**27**
RMSProp	0.5547	0.6735	0.2598	0.8667	0.3750	0.6941	12
AdaDelta	0.5992	0.5722	0.8740	0.3083	0.6916	0.6623	12
Adam	0.5951	0.6849	0.3937	0.8083	0.5000	0.6365	18
SGD	0.4049	0.3333	0.1575	0.6667	0.2139	0.4566	40
Adagrad	0.4494	0.4783	0.7795	0.1000	0.5928	0.2459	12

Learning Rate: 1e-6

9.4.4 EfficientNetB3

Table 16: EfficientNetB3 results at learning rate 1e-4

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
RMSProp	**0.8462**	**1.0000**	**0.7008**	**1.0000**	**0.8241**	**0.9841**	**41**
Adam	0.8988	0.9636	0.8346	0.9667	0.8945	0.9787	42
SGD	0.8623	0.9697	0.7559	0.9750	0.8496	0.9696	47
Adamax	0.8462	0.9588	0.7323	0.9667	0.8304	0.9589	50
AdaDelta	0.5668	0.9545	0.1654	0.9917	0.2819	0.8338	11
Adagrad	0.4939	1.0000	0.0157	1.0000	0.0310	0.6686	29

Learning Rate: 1e-4

Table 17: EfficientNetB3 results at learning rate 1e-5

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
RMSProp	**0.8502**	**0.9245**	**0.7717**	**0.9333**	**0.8412**	**0.9489**	**50**
AdaDelta	0.5668	0.9545	0.1654	0.9917	0.2819	0.8531	15
SGD	0.7166	0.6610	0.9213	0.5000	0.7697	0.7829	50
Adamax	0.6964	0.6494	0.8898	0.4917	0.7508	0.7335	37
Adam	0.5628	0.5497	0.8268	0.2833	0.6604	0.5308	43
Adagrad	0.4777	0.3750	0.0236	0.9583	0.0444	0.3934	28

Learning Rate: 1e-5

Table 18: EfficientNetB3 results at learning rate 1e-6

Optimizer	Acc	Prec	Rec	Spec	F1	AUC	Ep
Adamax	0.4858	0.0000	0.0000	1.0000	0.0000	0.8089	21
RMSProp	**0.6032**	**0.8718**	**0.2677**	**0.9583**	**0.4096**	**0.7854**	**20**
AdaDelta	0.4534	0.3750	0.0945	0.8333	0.1509	0.4241	11
Adam	0.4130	0.3676	0.1969	0.6417	0.2564	0.3281	14
Adagrad	0.3603	0.2817	0.1575	0.5750	0.2020	0.3001	50
SGD	0.4413	0.0000	0.0000	0.9083	0.0000	0.1893	33

Learning Rate: 1e-6

9.5 Complete 72-Run Results Table

Table 19: Complete 72-run results table sorted by AUC-ROC descending.

Rk	Run Name	Arch	Opt	LR	Acc	Prec	Rec	Spec	F1	AUC	Ep
1	VGG19_Adam_LR1e-04	VGG19	Adam	1e-4	0.7733	1.0000	0.5591	1.0000	0.7172	0.9995	26
2	VGG19_Adamax_LR1e-04	VGG19	Adamax	1e-4	0.9717	0.9918	0.9528	0.9917	0.9719	0.9995	50
3	DenseNet121_Adam_LR1e-04	DenseNet121	Adam	1e-4	0.9676	1.0000	0.9370	1.0000	0.9675	0.9991	27
4	DenseNet121_RMSProp_LR1e-04	DenseNet121	RMSProp	1e-4	0.9595	1.0000	0.9213	1.0000	0.9590	0.9990	24
5	DenseNet121_Adamax_LR1e-04	DenseNet121	Adamax	1e-4	0.9231	0.8699	1.0000	0.8417	0.9304	0.9977	23
6	VGG19_RMSProp_LR1e-04	VGG19	RMSProp	1e-4	0.8178	1.0000	0.6457	1.0000	0.7847	0.9949	21
7	ResNet50_RMSProp_LR1e-04	ResNet50	RMSProp	1e-4	0.9514	1.0000	0.9055	1.0000	0.9504	0.9943	40
8	VGG19_RMSProp_LR1e-05	VGG19	RMSProp	1e-5	0.9312	0.9583	0.9055	0.9583	0.9312	0.9933	50
9	DenseNet121_SGD_LR1e-04	DenseNet121	SGD	1e-4	0.8178	1.0000	0.6457	1.0000	0.7847	0.9885	30
10	ResNet50_SGD_LR1e-04	ResNet50	SGD	1e-4	0.9150	0.9907	0.8425	0.9917	0.9106	0.9869	35
11	ResNet50_Adam_LR1e-04	ResNet50	Adam	1e-4	0.9352	0.9912	0.8819	0.9917	0.9333	0.9864	31
12	EfficientNetB3_RMSProp_LR1e-04	EfficientNetB3	RMSProp	1e-4	0.8462	1.0000	0.7008	1.0000	0.8241	0.9841	41
13	VGG19_SGD_LR1e-04	VGG19	SGD	1e-4	0.8785	1.0000	0.7638	1.0000	0.8661	0.9838	50
14	ResNet50_Adamax_LR1e-04	ResNet50	Adamax	1e-4	0.9150	0.9818	0.8504	0.9833	0.9114	0.9815	44
15	EfficientNetB3_Adam_LR1e-04	EfficientNetB3	Adam	1e-4	0.8988	0.9636	0.8346	0.9667	0.8945	0.9787	42
16	ResNet50_Adam_LR1e-05	ResNet50	Adam	1e-5	0.8704	1.0000	0.7480	1.0000	0.8559	0.9768	50
17	ResNet50_SGD_LR1e-05	ResNet50	SGD	1e-5	0.9190	0.9496	0.8898	0.9500	0.9187	0.9722	32
18	EfficientNetB3_SGD_LR1e-04	EfficientNetB3	SGD	1e-4	0.8623	0.9697	0.7559	0.9750	0.8496	0.9696	47
19	DenseNet121_RMSProp_LR1e-05	DenseNet121	RMSProp	1e-5	0.9069	0.9407	0.8740	0.9417	0.9061	0.9619	37
20	EfficientNetB3_Adamax_LR1e-04	EfficientNetB3	Adamax	1e-4	0.8462	0.9588	0.7323	0.9667	0.8304	0.9589	50
21	ResNet50_RMSProp_LR1e-05	ResNet50	RMSProp	1e-5	0.8340	0.9574	0.7087	0.9667	0.8145	0.9510	36
22	EfficientNetB3_RMSProp_LR1e-05	EfficientNetB3	RMSProp	1e-5	0.8502	0.9245	0.7717	0.9333	0.8412	0.9489	50
23	DenseNet121_Adam_LR1e-05	DenseNet121	Adam	1e-5	0.8016	0.7600	0.8976	0.7000	0.8231	0.9284	50
24	DenseNet121_Adagrad_LR1e-04	DenseNet121	Adagrad	1e-4	0.4939	1.0000	0.0157	1.0000	0.0310	0.8734	25
25	ResNet50_Adagrad_LR1e-04	ResNet50	Adagrad	1e-4	0.7854	0.7372	0.9055	0.6583	0.8127	0.8718	50
26	VGG19_Adamax_LR1e-06	VGG19	Adamax	1e-6	0.8016	0.8482	0.7480	0.8583	0.7950	0.8713	11
27	EfficientNetB3_AdaDelta_LR1e-05	EfficientNetB3	AdaDelta	1e-5	0.5668	0.9545	0.1654	0.9917	0.2819	0.8531	15
28	VGG19_Adagrad_LR1e-05	VGG19	Adagrad	1e-5	0.6154	0.9211	0.2756	0.9750	0.4242	0.8491	38
29	ResNet50_Adamax_LR1e-05	ResNet50	Adamax	1e-5	0.7368	0.7214	0.7953	0.6750	0.7566	0.8366	38
30	ResNet50_AdaDelta_LR1e-06	ResNet50	AdaDelta	1e-6	0.5142	0.5142	1.0000	0.0000	0.6791	0.8350	11
31	EfficientNetB3_AdaDelta_LR1e-04	EfficientNetB3	AdaDelta	1e-4	0.5668	0.9545	0.1654	0.9917	0.2819	0.8338	11
32	EfficientNetB3_Adamax_LR1e-06	EfficientNetB3	Adamax	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.8089	21
33	ResNet50_Adagrad_LR1e-06	ResNet50	Adagrad	1e-6	0.5668	0.5435	0.9843	0.1250	0.7003	0.7961	25
34	EfficientNetB3_RMSProp_LR1e-06	EfficientNetB3	RMSProp	1e-6	0.6032	0.8718	0.2677	0.9583	0.4096	0.7854	20
35	ResNet50_SGD_LR1e-06	ResNet50	SGD	1e-6	0.7328	0.7440	0.7323	0.7333	0.7381	0.7844	41
36	EfficientNetB3_SGD_LR1e-05	EfficientNetB3	SGD	1e-5	0.7166	0.6610	0.9213	0.5000	0.7697	0.7829	50
37	ResNet50_RMSProp_LR1e-06	ResNet50	RMSProp	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.7809	11
38	DenseNet121_Adamax_LR1e-06	DenseNet121	Adamax	1e-6	0.6559	0.8000	0.4409	0.8833	0.5685	0.7663	27
39	EfficientNetB3_Adamax_LR1e-05	EfficientNetB3	Adamax	1e-5	0.6964	0.6494	0.8898	0.4917	0.7508	0.7335	37
40	ResNet50_Adamax_LR1e-06	ResNet50	Adamax	1e-6	0.6235	0.6889	0.4882	0.7667	0.5714	0.7269	13
41	VGG19_Adamax_LR1e-05	VGG19	Adamax	1e-5	0.6194	0.6813	0.4882	0.7583	0.5688	0.7215	35
42	DenseNet121_AdaDelta_LR1e-05	DenseNet121	AdaDelta	1e-5	0.6235	0.7833	0.3701	0.8917	0.5027	0.7146	11
43	VGG19_Adam_LR1e-05	VGG19	Adam	1e-5	0.6073	0.6442	0.5276	0.6917	0.5801	0.7117	50
44	VGG19_Adagrad_LR1e-04	VGG19	Adagrad	1e-4	0.4858	0.0000	0.0000	1.0000	0.0000	0.6997	25
45	DenseNet121_RMSProp_LR1e-06	DenseNet121	RMSProp	1e-6	0.5547	0.6735	0.2598	0.8667	0.3750	0.6941	12
46	EfficientNetB3_Adagrad_LR1e-04	EfficientNetB3	Adagrad	1e-4	0.4939	1.0000	0.0157	1.0000	0.0310	0.6686	29
47	VGG19_Adagrad_LR1e-06	VGG19	Adagrad	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.6667	11
48	VGG19_AdaDelta_LR1e-05	VGG19	AdaDelta	1e-5	0.5223	0.5187	0.9843	0.0333	0.6793	0.6634	11
49	DenseNet121_AdaDelta_LR1e-06	DenseNet121	AdaDelta	1e-6	0.5992	0.5722	0.8740	0.3083	0.6916	0.6623	12
50	DenseNet121_Adam_LR1e-06	DenseNet121	Adam	1e-6	0.5951	0.6849	0.3937	0.8083	0.5000	0.6365	18
51	DenseNet121_SGD_LR1e-05	DenseNet121	SGD	1e-5	0.5830	0.6429	0.4252	0.7500	0.5118	0.6067	50
52	VGG19_Adam_LR1e-06	VGG19	Adam	1e-6	0.4858	0.0000	0.0000	1.0000	0.0000	0.5520	11
53	VGG19_AdaDelta_LR1e-06	VGG19	AdaDelta	1e-6	0.5628	0.6234	0.3780	0.7583	0.4706	0.5505	14
54	EfficientNetB3_Adam_LR1e-05	EfficientNetB3	Adam	1e-5	0.5628	0.5497	0.8268	0.2833	0.6604	0.5308	43
55	ResNet50_Adagrad_LR1e-05	ResNet50	Adagrad	1e-5	0.4858	0.0000	0.0000	1.0000	0.0000	0.5074	12
56	DenseNet121_Adamax_LR1e-05	DenseNet121	Adamax	1e-5	0.4777	0.4872	0.2992	0.6667	0.3707	0.4989	49
57	DenseNet121_Adagrad_LR1e-05	DenseNet121	Adagrad	1e-5	0.4696	0.4778	0.3386	0.6083	0.3963	0.4711	27
58	DenseNet121_SGD_LR1e-06	DenseNet121	SGD	1e-6	0.4049	0.3333	0.1575	0.6667	0.2139	0.4566	40
59	DenseNet121_AdaDelta_LR1e-04	DenseNet121	AdaDelta	1e-4	0.4291	0.4444	0.4409	0.4167	0.4427	0.4281	33
60	VGG19_SGD_LR1e-05	VGG19	SGD	1e-5	0.5385	0.5279	0.9685	0.0833	0.6833	0.4278	22
61	EfficientNetB3_AdaDelta_LR1e-06	EfficientNetB3	AdaDelta	1e-6	0.4534	0.3750	0.0945	0.8333	0.1509	0.4241	11
62	ResNet50_AdaDelta_LR1e-05	ResNet50	AdaDelta	1e-5	0.5223	0.5184	1.0000	0.0167	0.6828	0.3954	12
63	EfficientNetB3_Adagrad_LR1e-05	EfficientNetB3	Adagrad	1e-5	0.4777	0.3750	0.0236	0.9583	0.0444	0.3934	28
64	VGG19_SGD_LR1e-06	VGG19	SGD	1e-6	0.4939	0.5041	0.9606	0.0000	0.6612	0.3815	11
65	VGG19_RMSProp_LR1e-06	VGG19	RMSProp	1e-6	0.4332	0.4652	0.6850	0.1667	0.5541	0.3577	35
66	ResNet50_Adam_LR1e-06	ResNet50	Adam	1e-6	0.4858	0.5000	0.8976	0.0500	0.6423	0.3573	30
67	VGG19_AdaDelta_LR1e-04	VGG19	AdaDelta	1e-4	0.5142	0.5145	0.9764	0.0250	0.6739	0.3490	12
68	EfficientNetB3_Adam_LR1e-06	EfficientNetB3	Adam	1e-6	0.4130	0.3676	0.1969	0.6417	0.2564	0.3281	14
69	EfficientNetB3_Adagrad_LR1e-06	EfficientNetB3	Adagrad	1e-6	0.3603	0.2817	0.1575	0.5750	0.2020	0.3001	50
70	DenseNet121_Adagrad_LR1e-06	DenseNet121	Adagrad	1e-6	0.4494	0.4783	0.7795	0.1000	0.5928	0.2459	12
71	EfficientNetB3_SGD_LR1e-06	EfficientNetB3	SGD	1e-6	0.4413	0.0000	0.0000	0.9083	0.0000	0.1893	33
72	ResNet50_AdaDelta_LR1e-04	ResNet50	AdaDelta	1e-4	0.4818	0.0000	0.0000	0.9917	0.0000	0.1554	11

9.6 Source Code

src/preprocessing.py

Listing 1: preprocessing.py : Data Preprocessing Pipeline

```
"""
preprocessing.py

Prepares the STRAMPN ovarian cancer dataset for training:
  - Inventories raw images and verifies expected counts
  - Performs a stratified train / val / test split (52.5% / 22.5% / 25%)
  - Copies files into dataset/processed/{split}/{class}/ directories
  - Prints a verification report and checks for cross-split leakage

Run directly:  python src/preprocessing.py
"""

import os
import shutil
import collections
from pathlib import Path

from sklearn.model_selection import train_test_split


# Directory layout

# Resolve paths relative to this file's location so the script works from
_PROJECT_ROOT = Path(__file__).resolve().parent.parent

_RAW_BASE = _PROJECT_ROOT / "dataset" / "raw" / "STRAMPN Dataset" / "Dataset"
_RAW_DIRS = {
    "cancer":    _RAW_BASE / "Ovarian_Cancer",
    "no_cancer": _RAW_BASE / "Ovarian_Non_Cancer",
}

_PROCESSED_BASE = _PROJECT_ROOT / "dataset" / "processed"

_EXPECTED_COUNTS = {"cancer": 481, "no_cancer": 506}
_EXPECTED_TOTAL  = 987


# Helpers

def _all_files(directory: Path) -> list[Path]:
    """Return a sorted list of all files (non-recursive) in directory."""
    return sorted(p for p in directory.iterdir() if p.is_file())


def _copy_file(src: Path, dst_dir: Path) -> str:
    """Copy src to dst_dir/src.name. Returns 'copied' or 'skipped'."""
    dst = dst_dir / src.name
    if dst.exists():
        return "skipped"
    shutil.copy2(src, dst)
    return "copied"


# Main function
def run_preprocessing(random_seed: int = 42) -> None:
    """
    Full preprocessing pipeline: inventory -> split -> copy -> verify.

    Args:
        random_seed: Controls the stratified split for reproducibility.
                     Must be identical to the seed used in all 84 training runs.
    """

    # Step 1 - Inventory
    print("-" * 60)
    print("STEP 1 - RAW DATASET INVENTORY")
    print("-" * 60)

    all_pairs: list[tuple[Path, str]] = []  # (filepath, label)

    for label, raw_dir in _RAW_DIRS.items():
        if not raw_dir.exists():
            raise FileNotFoundError(
                f"Raw directory not found: {raw_dir}\n"
                "Download the STRAMPN dataset and place it at:\n"
                f"  {_RAW_BASE}"
            )

        files = _all_files(raw_dir)
        non_jpg = [f for f in files if f.suffix.lower() != ".jpg"]

        print(f"\n  [{label}]  path: {raw_dir}")
        print(f"    Total files : {len(files)}")

        if non_jpg:
            print(f"    [WARNING] Non
```

```
                -JPG files found ({
                len(non_jpg)}):")
            for f in non_jpg:
                print(f"        {f.
                    name}")
        else:
            print(f"    All files    :
                 .jpg [ok]")

        jpg_files - [f for f in files
             if f.suffix.lower() -- "
            .jpg"]
        expected  - _EXPECTED_COUNTS[
            label]
        match_sym - "[ok]" if len(
            jpg_files) -- expected
            else "[x] (expected {ex
            pected})"
        print(f"    JPG count    : {
            len(jpg_files)}  {
            match_sym}")

        all_pairs.extend((f, label)
            for f in jpg_files)

    total - len(all_pairs)
    total_sym - "[ok]" if total --
        _EXPECTED_TOTAL else f"[x] (e
        xpected {_EXPECTED_TOTAL})"
    print(f"\n  Total images across
        both classes: {total}  {
        total_sym}")

    # Step 2 - Stratified Split
    print("\n" + "-" * 60)
    print("STEP 2 - STRATIFIED SPLIT"
        )
    print("-" * 60)

    filepaths - [p for p, _ in
        all_pairs]
    labels    - [l for _, l in
        all_pairs]

    # First split: 75% train+val, 25%
         test
    paths_trainval, paths_test,
        labels_trainval, labels_test
        - train_test_split(
        filepaths, labels,
        test_size-0.25,
        random_state-random_seed,
        stratify-labels,
    )

    # Second split: 70% train, 30%
        val  (of the 75% slice ->
        ~52.5% / 22.5% overall)
    paths_train, paths_val,
        labels_train, labels_val -
        train_test_split(
        paths_trainval,
            labels_trainval,
        test_size-0.30,
        random_state-random_seed,
        stratify-labels_trainval,
    )

    splits - {
        "train": (paths_train,
            labels_train),
        "val":   (paths_val,
            labels_val),
        "test":  (paths_test,
            labels_test),
    }

    print(f"\n  Seed: {random_seed}")
    print(f"  {'Split':<8}  {'cancer
        ':>8}  {'no_cancer':>10}  {'
        total':>7}")
    print(f"  {'-'*8}  {'-'*8}  {'-
        '*10}  {'-'*7}")
    for split_name, (paths, lbls) in
        splits.items():
        c  - lbls.count("cancer")
        nc - lbls.count("no_cancer")
        print(f"  {split_name:<8}  {c
            :>8}  {nc:>10}  {len(lbls
            ):>7}")
    print(f"  {'total':<8}  {labels.
        count('cancer'):>8}  {labels.
        count('no_cancer'):>10}  {
        total:>7}")

    # Step 3 - Copy Files
    print("\n" + "-" * 60)
    print("STEP 3 - COPYING FILES TO
        dataset/processed/")
    print("-" * 60)

    for split_name, (paths, lbls) in
        splits.items():
        copied_total  - 0
        skipped_total - 0

        for src, label in zip(paths,
            lbls):
            dst_dir - _PROCESSED_BASE
                 / split_name / label
            dst_dir.mkdir(parents-
                True, exist_ok-True)
            result - _copy_file(src,
                dst_dir)
            if result -- "copied":
                copied_total  += 1
            else:
                skipped_total += 1

        print(
            f"  {split_name:<6}
                copied: {copied_total
                :>4}   "
            f"skipped (already exist)
                : {skipped_total:>4}"
        )

    # Step 4 - Verification Report

    print("\n" + "-" * 60)
    print("STEP 4 - VERIFICATION
        REPORT")
    print("-" * 60)

    class_names - ["cancer", "
        no_cancer"]
    grand_total - 0

    print(f"\n  {'Split':<8}  {'Class
        ':<12}  {'Count':>6}  {'% of
        total':>10}")
    print(f"  {'-'*8}  {'-'*12}  {'-
        '*6}  {'-'*10}")

    for split_name in ["train", "val"
        , "test"]:
```

```
        for cls in class_names:
            cls_dir = _PROCESSED_BASE / split_name / cls
            n = len(list(cls_dir.glob("*.jpg"))) if cls_dir.exists() else 0
            pct = n / total * 100
            grand_total += n
            print(f"  {split_name:<8} {cls:<12}  {n:>6}  {pct:>9.1f}%")

    print(f"\n  Grand total in processed/: {grand_total}  "
          f"({'[ok] matches raw' if grand_total == total else '[x] MISMATCH with raw'})")

    # Cross-split leakage check - compare stem names across splits
    print("\n  Checking for cross-split filename leakage.")
    split_stems: dict[str, set[str]] = {}
    for split_name in ["train", "val", "test"]:
        stems: set[str] = set()
        for cls in class_names:
            cls_dir = _PROCESSED_BASE / split_name / cls
            if cls_dir.exists():
                stems.update(p.stem for p in cls_dir.glob("*.jpg"))
        split_stems[split_name] = stems

    violations: list[str] = []
    split_list = list(split_stems.items())
    for i in range(len(split_list)):
        for j in range(i + 1, len(split_list)):
            a_name, a_stems = split_list[i]
            b_name, b_stems = split_list[j]
            overlap = a_stems & b_stems
            if overlap:
                violations.append(
                    f"    [x] {a_name} intersect {b_name}: {len(overlap)} shared filename(s): "
                    + ", ".join(sorted(overlap)[:5])
                    + ("..." if len(overlap) > 5 else "")
                )

    print()
    if violations:
        print("  [WARNING] INTEGRITY VIOLATIONS FOUND:")
        for v in violations:
            print(v)
    else:
        print("  PREPROCESSING COMPLETE - No integrity violations found")

    print("\n" + "-" * 60)


# Entry point

if __name__ == "__main__":
    run_preprocessing()
```

src/model_builder.py

Listing 2: model_builder.py : Model Architecture Builder

```
"""
model_builder.py

Builds compiled Keras transfer-learning models for the 72-simulation experiment matrix.
"""

import tensorflow as tf
from tensorflow.keras import Model
from tensorflow.keras.applications import VGG19, EfficientNetB3, ResNet50, DenseNet121
from tensorflow.keras.layers import (
    GlobalAveragePooling2D, Dense, Dropout, BatchNormalization,
)
from tensorflow.keras.metrics import Precision, Recall, AUC
from tensorflow.keras.optimizers import (
    Adam, Adagrad, Adamax, Adadelta, SGD, RMSprop,
)

# Architecture registry

_ARCHITECTURE_REGISTRY = {
    "VGG19":          VGG19,
    "EfficientNetB3": EfficientNetB3,
    "ResNet50":       ResNet50,
    "DenseNet121":    DenseNet121,
}

# Optimizer registry
def _make_optimizer(name: str, learning_rate: float):
    """Instantiate an optimizer by name with the given learning rate."""
    registry = {
        "Adam":     lambda lr: Adam(learning_rate=lr),
        "Adagrad":  lambda lr: Adagrad(learning_rate=lr),
        "Adamax":   lambda lr: Adamax(learning_rate=lr),
        "AdaDelta": lambda lr: Adadelta(learning_rate=lr),
```

```
        "SGD":      lambda lr: SGD(learning_rate=lr, momentum=0.9),
        "RMSProp":  lambda lr: RMSprop(learning_rate=lr),
    }
    if name not in registry:
        raise ValueError(
            f"Unknown optimizer '{name}'. "
            f"Choose from: {sorted(registry.keys())}"
        )
    return registry[name](learning_rate)


# Model builder
def build_model(
    architecture_name: str,
    optimizer_name: str,
    learning_rate: float,
    input_shape: tuple = (256, 256, 3),
) -> Model:
    """
    Build and compile a transfer-learning model for binary ovarian cancer classification.

    Args:
        architecture_name: One of 'VGG19', 'EfficientNetB3', 'ResNet50', 'DenseNet121'.
        optimizer_name:    One of 'Adam', 'Adagrad', 'Adamax', 'AdaDelta',
                           'SGD', 'RMSProp'.
        learning_rate:     Floating-point learning rate (e.g. 1e-4).
        input_shape:       HWC tuple; must match the generators (default 256x256x3).

    Returns:
        A compiled tf.keras.Model ready for model.fit().
    """
    if architecture_name not in _ARCHITECTURE_REGISTRY:
        raise ValueError(
            f"Unknown architecture '{architecture_name}'. "
            f"Choose from: {sorted(_ARCHITECTURE_REGISTRY.keys())}"
        )

    # Base model (frozen ImageNet weights)
    base_cls = _ARCHITECTURE_REGISTRY[architecture_name]
    base_model = base_cls(
        include_top=False,
        weights="imagenet",
        input_shape=input_shape,
    )
    base_model.trainable = False

    # Custom classification head
    x = base_model.output
    x = GlobalAveragePooling2D()(x)

    x = Dense(256, activation="relu")(x)
    x = BatchNormalization()(x)
    x = Dropout(0.6)(x)

    x = Dense(128, activation="relu")(x)
    x = BatchNormalization()(x)
    x = Dropout(0.4)(x)

    x = Dense(64, activation="relu")(x)
    x = BatchNormalization()(x)
    x = Dropout(0.3)(x)

    output = Dense(1, activation="sigmoid")(x)

    model = Model(inputs=base_model.input, outputs=output)

    # Compile
    optimizer = _make_optimizer(optimizer_name, learning_rate)
    model.compile(
        optimizer=optimizer,
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            Precision(name="precision"),
            Recall(name="recall"),
            AUC(name="auc"),
        ],
    )

    return model
```

src/train.py

Listing 3: train.py : Model Training Loop

```
"""
train.py

Runs the training loop for a single model configuration (one of the 72 runs).
Called once per simulation from run_all.py or 02_experiments.ipynb.

Each run writes the following artifacts to results/{run_name}/:

  - best_model.keras                  - best checkpoint by val_auc
  - training_log.csv                  - per-epoch metrics
  - {run_name}_acc_loss.png           - accuracy and loss curves
```

```
  - {run_name}_auc_roc.png                    - AUC-ROC curve
  - confusion_matrices/{run_name}.png         - confusion matrix heatmap
  - (metrics row appended to master CSV by the caller)
"""

import pathlib

import tensorflow as tf
from tensorflow.keras.callbacks import (
    CSVLogger,
    EarlyStopping,
    ModelCheckpoint,
    ReduceLROnPlateau,
)

from src.evaluate import evaluate_model
from src.visualize import plot_accuracy_loss, plot_auc_roc


def train_model(
    model: tf.keras.Model,
    train_generator,
    val_generator,
    test_generator,
    architecture: str,
    optimizer_name: str,
    learning_rate: float,
    results_base_dir,
    epochs: int = 50,
) -> dict:
    """
    Train one model configuration, evaluate on the test set, and save all artifacts.

    Args:
        model:             Compiled Keras model from model_builder.build_model().
        train_generator:   Training data generator (shared, not re-instantiated).
        val_generator:     Validation data generator.
        test_generator:    Test data generator.
        architecture:      Base architecture name (e.g. 'ResNet50').
        optimizer_name:    Optimizer name (e.g. 'Adam').
        learning_rate:     Learning rate float (e.g. 1e-4).
        results_base_dir:  Path-like; run output written to results_base_dir/run_name/.
        epochs:            Maximum number of training epochs.

    Returns:
        metrics dict as returned by evaluate_model (one row of the master CSV).
    """
    results_base_dir = pathlib.Path(results_base_dir)

    # Step 1 - Setup
    lr_str   = f"{learning_rate:.0e}"                    # e.g. '1e-04'
    run_name = f"{architecture}_{optimizer_name}_LR{lr_str}"
    run_dir  = results_base_dir / run_name
    run_dir.mkdir(parents=True, exist_ok=True)

    print("=" * 60)
    print(f"TRAINING: {run_name}")
    print(f"Architecture : {architecture}")
    print(f"Optimizer    : {optimizer_name}")
    print(f"Learning Rate: {learning_rate}")
    print("=" * 60)

    # Step 2 - Callbacks
    callbacks = [
        ModelCheckpoint(
            filepath=str(run_dir / "best_model.keras"),
            monitor="val_auc",
            mode="max",
            save_best_only=True,
            verbose=0,
        ),
        EarlyStopping(
            monitor="val_auc",
            patience=10,
            mode="max",
            restore_best_weights=True,
            min_delta=0.001,
            verbose=1,
        ),
        CSVLogger(
            filename=str(run_dir / "training_log.csv"),
            append=False,
        ),
        ReduceLROnPlateau(
            monitor="val_loss",
            factor=0.5,
            patience=5,
            min_lr=1e-7,
            verbose=1,
        ),
    ]

    # Step 3 - Training
    history = model.fit(
        train_generator,
        validation_data=val_generator,
        epochs=epochs,
        callbacks=callbacks,
        verbose=1,
    )

    actual_epochs  = len(history.history["loss"])
    best_val_auc   = max(history.history["val_auc"])
    print(f"\nTraining complete - epochs run: {actual_epochs} /
```

```
            {epochs}  |  best val_auc: {
            best_val_auc:.4f}")
116
117     # Step 4 - Evaluation and
            Visualization
118     metrics - evaluate_model(
119         model-model,
120         test_generator-test_generator
                ,
121         run_name-run_name,
122         output_dir-str(run_dir),
123     )
124
125     # saves {run_name}_acc_loss.png
            into run_dir
126     plot_accuracy_loss(
127         history-history,
128         run_name-run_name,
129         output_dir-str(run_dir),
130     )
131
132     # saves {run_name}_auc_roc.png
            into run_dir
133     plot_auc_roc(
134         model-model,
135         test_generator-test_generator
                ,
136         run_name-run_name,
137         output_dir-str(run_dir),
138     )
139
140     metrics["epochs_run"] -
            actual_epochs
141
142     # Step 5 - Cleanup
143     tf.keras.backend.clear_session()
144
145     print(f"\nCOMPLETED: {run_name}")
146     print("-" * 60)
147
148     return metrics
```

src/evaluate.py

Listing 4: evaluate.py : Model Evaluation

```
1  """
2  evaluate.py
3
4  Evaluates a trained model on the held
       -out test set and persists per-
       run metrics.
5  """
6
7  import os
8  import csv
9  import math
10
11 import numpy as np
12 from sklearn.metrics import (
13     confusion_matrix,
14     precision_score,
15     recall_score,
16     f1_score,
17     roc_auc_score,
18     accuracy_score,
19 )
20
21 from src.visualize import
       plot_confusion_matrix
22
23
24 def evaluate_model(
25     model,
26     test_generator,
27     run_name: str,
28     output_dir: str,
29 ) -> dict:
30     """
31     Evaluate a trained model and
           persist all metrics and plots
           .
32
33     Args:
34         model:          Trained tf.
               keras.Model.
35         test_generator: Keras data
               generator for the test
               set.
36                         Must NOT
                               shuffle (
                               shuffle-
                               False) so
                                labels
                               stay
                               aligned.
37         run_name:       Unique
               identifier for this run,
               e.g.
38                         '
                               ResNet50_Adam_LR1e
                               -4'.
                               Used as
                               the
                               filename
                               stem for
39                         all saved
                               artefacts
                               .
40         output_dir:     Root results/
                directory.  Sub-
               directories
41                         (metrics/,
                               confusion_matrices
                               /) are
                               created
                               if absent
                               .
42
43     Returns:
44         dict with keys:
45             run_name, architecture,
                   optimizer,
                   learning_rate,
46             accuracy, precision,
                   recall, specificity,
                   f1, auc
47     """
48     # Prediction
49     # Reset generator so prediction
           starts at the first batch.
50     test_generator.reset()
51     y_prob - model.predict(
           test_generator, verbose-0).
           ravel()
52     y_pred - (y_prob >= 0.5).astype(
           int)
53     y_true - test_generator.classes
54
55     # Scalar Metrics
56     accuracy  - accuracy_score(y_true
           , y_pred)
57     precision - precision_score(
           y_true, y_pred, zero_division
           -0)
58     recall    - recall_score(y_true,
```

```
        y_pred, zero_division=0)
    f1        = f1_score(y_true,
        y_pred, zero_division=0)
    auc       = roc_auc_score(y_true,
         y_prob)

    # Specificity: TN / (TN + FP)
    cm = confusion_matrix(y_true,
        y_pred)
    tn, fp = cm[0, 0], cm[0, 1]
    specificity = tn / (tn + fp) if (
        tn + fp) > 0 else 0.0

    # Parse run_name into components
    # Expected format: {Architecture}
        _{Optimizer}_LR{lr}  e.g.
        ResNet50_Adam_LR1e-4
    parts = run_name.split("_")
    architecture   = parts[0] if len(
        parts) > 0 else run_name
    optimizer_name = parts[1] if len(
        parts) > 1 else ""
    learning_rate  = parts[2].replace
        ("LR", "") if len(parts) > 2
        else ""

    # Confusion matrix plot
    cm_dir = os.path.join(output_dir,
         "confusion_matrices")
    os.makedirs(cm_dir, exist_ok=True
        )
    plot_confusion_matrix(
        cm=cm,
        run_name=run_name,
        class_names=list(
            test_generator.
            class_indices.keys()),
        output_dir=cm_dir,
    )

    # Save metrics CSV (one row per
        run)
    metrics_dir = os.path.join(
        output_dir, "metrics")
    os.makedirs(metrics_dir, exist_ok
        =True)
    csv_path = os.path.join(
        metrics_dir, f"{run_name}.csv
        ")

    metrics = {
        "run_name":     run_name,
        "architecture": architecture,
        "optimizer":
            optimizer_name,
        "learning_rate":
            learning_rate,
        "accuracy":     round(
            accuracy,    4),
        "precision":    round(
            precision,   4),
        "recall":       round(recall,
                    4),
        "specificity":  round(
            specificity, 4),
        "f1":           round(f1,
                     4),
        "auc":          round(auc,
                    4),
    }

    with open(csv_path, "w", newline=
        "") as fh:
        writer = csv.DictWriter(fh,
            fieldnames=list(metrics.
            keys()))
        writer.writeheader()
        writer.writerow(metrics)

    print(
        f"[evaluate] {run_name} | "
        f"acc={accuracy:.4f}  prec={
            precision:.4f}  rec={
            recall:.4f}  "
        f"spec={specificity:.4f}  f1=
            {f1:.4f}  auc={auc:.4f}"
    )

    return metrics
```

src/visualize.py

Listing 5: visualize.py : Results Visualization

```
"""
visualize.py

Visualization utilities for the
    optimizer/learning-rate study.
    Each training run
produces three plot types:

  - Accuracy/loss curves  -> {run_dir
    }/{run_name}_acc_loss.png
  - AUC-ROC curves        -> {run_dir
    }/{run_name}_auc_roc.png
  - Confusion matrices    -> {run_dir
    }/confusion_matrices/{run_name
    }.png
"""

import os

import numpy as np
import matplotlib
matplotlib.use("Agg")  # non-
    interactive backend - safe for
    headless runs
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import roc_curve
    , auc as sklearn_auc

# Shared style Settings

_STYLE        = "seaborn-v0_8-
    whitegrid"
_FIG_DPI      = 150
_CANCER_COLOR = "#C0392B"   # red -
    cancer class
_NORMAL_COLOR = "#2980B9"   # blue -
    no_cancer class


# Accuracy / Loss curves

def plot_accuracy_loss(history,
    run_name: str, output_dir: str) ->
     None:
    """
    Plot training and validation
        accuracy + loss curves and
        save as PNG.

```

```
    Saves to: {output_dir}/{run_name}_acc_loss.png

    Args:
        history:    tf.keras.callbacks.History object (or its .history dict).
        run_name:   Stem used for the output filename.
        output_dir: Directory in which the figure is saved (typically the run directory).
    """
    os.makedirs(output_dir, exist_ok=True)

    h = history.history if hasattr(history, "history") else history
    epochs = range(1, len(h["accuracy"]) + 1)

    with plt.style.context(_STYLE):
        fig, (ax_acc, ax_loss) = plt.subplots(1, 2, figsize=(12, 4))
        fig.suptitle(run_name, fontsize=11, fontweight="bold")

        # Accuracy subplot
        ax_acc.plot(epochs, h["accuracy"],     label="Train Accuracy", linewidth=1.8)
        ax_acc.plot(epochs, h["val_accuracy"], label="Val Accuracy", linewidth=1.8, linestyle="--")
        ax_acc.set_title("Accuracy")
        ax_acc.set_xlabel("Epoch")
        ax_acc.set_ylabel("Accuracy")
        ax_acc.legend()
        ax_acc.set_ylim(0, 1)

        # Loss subplot
        ax_loss.plot(epochs, h["loss"],     label="Train Loss", linewidth=1.8)
        ax_loss.plot(epochs, h["val_loss"], label="Val Loss",   linewidth=1.8, linestyle="--")
        ax_loss.set_title("Loss")
        ax_loss.set_xlabel("Epoch")
        ax_loss.set_ylabel("Binary Cross-Entropy")
        ax_loss.legend()

        fig.tight_layout()
        out_path = os.path.join(output_dir, f"{run_name}_acc_loss.png")
        fig.savefig(out_path, dpi=_FIG_DPI, bbox_inches="tight")
        plt.close(fig)

    print(f"[visualize] Accuracy/Loss curve saved -> {out_path}")


# AUC-ROC curve

def plot_auc_roc(model, test_generator, run_name: str, output_dir: str) -> None:
    """
    Compute the ROC curve and save the AUC-ROC plot as PNG.

    Saves to: {output_dir}/{run_name}_auc_roc.png

    Args:
        model:          Trained tf.keras.Model.
        test_generator: Test generator with shuffle=False.
        run_name:       Stem used for the output filename.
        output_dir:     Directory in which the figure is saved (typically the run directory).
    """
    os.makedirs(output_dir, exist_ok=True)

    test_generator.reset()
    y_prob = model.predict(test_generator, verbose=0).ravel()
    y_true = test_generator.classes

    fpr, tpr, _ = roc_curve(y_true, y_prob)
    roc_auc     = sklearn_auc(fpr, tpr)

    with plt.style.context(_STYLE):
        fig, ax = plt.subplots(figsize=(6, 5))
        ax.plot(fpr, tpr, color=_CANCER_COLOR, linewidth=2,
                label=f"AUC = {roc_auc:.4f}")
        ax.plot([0, 1], [0, 1], color="grey", linestyle="--", linewidth=1,
                label="Random Classifier")
        ax.set_xlabel("False Positive Rate")
        ax.set_ylabel("True Positive Rate")
        ax.set_title(f"ROC Curve — {run_name}", fontweight="bold")
        ax.legend(loc="lower right")
        ax.set_xlim(0, 1)
        ax.set_ylim(0, 1.02)

        fig.tight_layout()
        out_path = os.path.join(output_dir, f"{run_name}_auc_roc.png")
        fig.savefig(out_path, dpi=_FIG_DPI, bbox_inches="tight")
        plt.close(fig)
```

```
    print(f"[visualize] AUC-ROC curve saved -> {out_path}")


# Confusion matrix

def plot_confusion_matrix(
    cm: np.ndarray,
    run_name: str,
    class_names: list,
    output_dir: str,
) -> None:
    """
    Plot a seaborn heatmap of the confusion matrix and save as PNG.

    Saves to: {output_dir}/{run_name}.png  (output_dir is typically confusion_matrices/ under the run directory)

    Args:
        cm:           2x2 numpy confusion matrix from sklearn.
        run_name:     Stem used for the output filename.
        class_names:  List of class label strings, e.g. ['cancer', 'no_cancer'].
        output_dir:   Directory in which the figure is saved (typically {run_dir}/confusion_matrices/).
    """
    os.makedirs(output_dir, exist_ok=True)

    with plt.style.context(_STYLE):
        fig, ax = plt.subplots(figsize=(5, 4))

        # Annotate each cell with count and row percentage
        total_per_row = cm.sum(axis=1, keepdims=True)
        pct = cm / total_per_row.clip(min=1) * 100
        annot = np.array(
            [[f"{cm[i, j]}\n({pct[i, j]:.1f}%)" for j in range(cm.shape[1])]
             for i in range(cm.shape[0])]
        )

        sns.heatmap(
            cm,
            annot=annot,
            fmt="",
            cmap="Blues",
            xticklabels=class_names,
            yticklabels=class_names,
            linewidths=0.5,
            ax=ax,
            cbar_kws={"shrink": 0.75},
        )
        ax.set_xlabel("Predicted Label", fontsize=10)
        ax.set_ylabel("True Label", fontsize=10)
        ax.set_title(f"Confusion Matrix - {run_name}", fontweight="bold", fontsize=11)

        fig.tight_layout()
        out_path = os.path.join(output_dir, f"{run_name}.png")
        fig.savefig(out_path, dpi=_FIG_DPI, bbox_inches="tight")
        plt.close(fig)

    print(f"[visualize] Confusion matrix saved -> {out_path}")
```

src/run_all.py

Listing 6: run_all.py : Experiment Orchestrator

```
"""
run_all.py

Master orchestration script for all 72 simulation runs.
Builds the shared data pipeline once, then loops over every combination of:
  - 4 architectures : VGG19, EfficientNetB3, ResNet50, DenseNet121
  - 6 optimizers    : Adam, Adagrad, Adamax, AdaDelta, SGD, RMSProp
  - 3 learning rates: 1e-4, 1e-5, 1e-6

Run directly:  python src/run_all.py
"""

import itertools
import pathlib
import random

import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from src.model_builder import build_model
from src.train import train_model


# Configuration

RANDOM_SEED   - 42
EPOCHS        - 50
BATCH_SIZE    - 32
INPUT_SIZE    - (256, 256)

ARCHITECTURES  - ['VGG19', 'EfficientNetB3', 'ResNet50', 'DenseNet121']
OPTIMIZERS     - ['Adam', 'Adagrad', 'Adamax', 'AdaDelta', 'SGD', 'RMSProp']
LEARNING_RATES - [1e-4, 1e-5, 1e-6]


```

```
# Helpers

def _seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)


# Main

def run_all() -> None:
    _seed_everything(RANDOM_SEED)

    project_root  = pathlib.Path(__file__).resolve().parent.parent
    processed_dir = project_root / "dataset" / "processed"
    results_dir   = project_root / "results"
    summary_dir   = results_dir / "summary"
    summary_dir.mkdir(parents=True, exist_ok=True)

    # Section 2 - Shared Data Pipeline (constructed ONCE, reused across all 72)

    image_generatorFullAugment = ImageDataGenerator(
        samplewise_center=True,
        samplewise_std_normalization=True,
        rotation_range=30,
        width_shift_range=0.15,
        height_shift_range=0.15,
        shear_range=0.3,
        zoom_range=0.30,
        horizontal_flip=True,
        fill_mode="nearest",
    )

    image_generatorNoAugment = ImageDataGenerator(
        samplewise_center=True,
        samplewise_std_normalization=True,
    )

    train_generator = image_generatorFullAugment.flow_from_directory(
        str(processed_dir / "train"),
        target_size=INPUT_SIZE,
        batch_size=BATCH_SIZE,
        class_mode="binary",
        shuffle=True,
        seed=RANDOM_SEED,
    )

    val_generator = image_generatorFullAugment.flow_from_directory(
        str(processed_dir / "val"),
        target_size=INPUT_SIZE,
        batch_size=1,
        class_mode="binary",
        shuffle=True,
        seed=RANDOM_SEED,
    )

    test_generator = image_generatorNoAugment.flow_from_directory(
        str(processed_dir / "test"),
        target_size=INPUT_SIZE,
        batch_size=1,
        class_mode="binary",
        shuffle=False,
    )

    print(f"Train : {train_generator.samples} images")
    print(f"Val   : {val_generator.samples} images")
    print(f"Test  : {test_generator.samples} images")
    print(f"Classes: {train_generator.class_indices}")
    print()
    print(
        "[WARNING]  EXPERIMENTAL INTEGRITY NOTE: generators are constructed ONCE here "
        "and shared across all 72 runs. Do not re-initialize them inside the loop - "
        "doing so would reshuffle the training data differently per run and confound "
        "optimizer/LR comparisons."
    )

    # Section 3 - 72-Run Training Loop

    experiment_matrix = list(itertools.product(ARCHITECTURES, OPTIMIZERS, LEARNING_RATES))
    total_runs        = len(experiment_matrix)

    print(f"\nTOTAL RUNS: {total_runs}")
    print("Starting simulation.\n")

    all_metrics: list[dict] = []

    for counter, (arch, opt, lr) in enumerate(experiment_matrix, start=1):
        lr_str = f"{lr:.0e}"
        print(f"\n{'-' * 60}")
        print(f"  Run {counter:02d}/{total_runs}  |  {arch}  |  {opt}  |  LR={lr_str}")
        print(f"{'-' * 60}")

        model = build_model(
            architecture_name=arch,
            optimizer_name=opt,
            learning_rate=lr,
        )

        metrics = train_model(
            model=model,
            train_generator=train_generator,
            val_generator=val_generator,
```

```
            test_generator=test_generator,
            architecture=arch,
            optimizer_name=opt,
            learning_rate=lr,
            results_base_dir=results_dir,
            epochs=EPOCHS,
        )

        all_metrics.append(metrics)

        train_generator.reset()
        test_generator.reset()

    # Section 4 - Aggregate Results

    master_df = pd.DataFrame(all_metrics)
    summary_csv = summary_dir / "master_results.csv"
    master_df.to_csv(summary_csv, index=False)

    print(f"\nMaster results saved -> {summary_csv}")
    print(master_df.to_string())
    print(f"\nALL {total_runs} SIMULATIONS COMPLETE")


# Entry point

if __name__ == "__main__":
    run_all()
```